

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Системы управления базами данных»
на тему

FDNJ Автоматизированная информационная система отслеживания цен на
товары

Студент гр. 420604
Консультант

М. А. Иосько
А. И. Лаппо

Минск 2026

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

УТВЕРЖДАЮ
Заведующий кафедрой
« 01 » сентября 2026 г.

ЗАДАНИЕ
на курсовую работу
по дисциплине «Системы управления базами данных»

Обучающийся

я Иосько Михаил Алексеевич

Курс 3 Учебная группа 420604

Тема курсовой

работы: Автоматизированная информационная система

отслеживания цен на товары

Перечень подлежащих разработке вопросов или краткое содержание
расчетно-

пояснительной записки: Задачи, решаемые в проекте: авторизация

пользователей, добавление товаров для отслеживания, редактирование

отслеживаемых товаров для редоктирования, возможность отключить

уведомления, отслеживать товары по ссылке (автоматически подтянутся

фильтры со страницы поиска)

Содержание пояснительной записки:

Введение

1. Обзор исследуемой области

2. Проектирование структуры «Автоматизированная информационная система
отслеживания цен на товары»

3. Разработка «Автоматизированная информационная система отслеживания
цен на товары»

4. Руководство пользователя и контрольные примеры

Заключение

Список использованных источников

Приложение А (обязательное) Листинг основных функций программного
модуля

Схема базы данных программного комплекса (ПЛ) – формат А3, лист 1.

Примерный календарный график выполнения курсовой работы: _____

Описание предметной области. Анализ систем-аналогов. Проектирование
структуры название темы. Проектирование базы данных. Проектирование
приложения Постановка задачи – 01.10.26

Разработка «Автоматизированная информационная система отслеживания
цен на товары». Выбор и обоснование средств реализации.

Реализация базы данных. Применение интеллектуальных инструментов в
инженерном проектировании. Программная реализация проекта.

Руководство пользователя и контрольные примеры – 01.11.26

Оформление графического материала и пояснительной записки

Отладка реализуемого проекта. – 01.12.26

Дата выдачи задания 01.09.2026 г.

Срок сдачи законченной курсовой работы 01.12.2026 г.

Консультант курсовой
работы _____ А. И. Лаппо

Подпись обучающегося _____ М. А. Иосько

Дата 01.09.2026 г. _____

СОДЕРЖАНИЕ

Введение	5
1 Обзор исследуемой области	7
1.1 Описание предметной области	7
1.2 Обзор аналогов	9
1.3 Постановка задачи.....	12
2 Проектирование системы	18
2.1 Проектирование баз данных	18
2.2 Проектирование системы	26
3 Разработка системы	37
3.1 Выбор средств реализации	37
3.2 Реализация базы данных	39
3.3 Применение интеллектуальных инструментов в инженерном проек- тировании	47
3.4 Программная реализация системы	50
4 Руководство пользователя	59
4.1 Роль администратора	59
4.2 Роль пользователя	62
Заключение	70
Список использованных источников	71
Приложение А (обязательное) Листинг основных функций программного модуля	72

ВВЕДЕНИЕ

Продажа подержанных товаров в Беларуси в значительной мере переместилась на площадки частных объявлений. Крупнейшая из них – *Kufar.by*, где одновременно размещены десятки тысяч объявлений в каждой из популярных категорий. Покупатель, который ищет товар дешевле рыночной цены, вынужден по несколько раз в день вручную просматривать выдачу. Новые объявления появляются круглосуточно, а предложения с заниженной ценой нередко снимаются с публикации в течение нескольких часов. Кто открыл площадку позже, узнаёт о таком объявлении уже после сделки. Отсюда потребность в системе, которая следит за площадкой вместо человека и сообщает ему только о подходящих предложениях.

Внешне такая система сводится к интерфейсу: пользователь задаёт в *Telegram* товар и пороговую цену, бот присылает ссылки на объявления. Но на интерфейс приходится меньшая часть работы. Основная нагрузка связана с обработкой данных, которые поступают без перерыва. Площадка опрашивается каждые несколько минут, и каждый полученный ответ сопоставляется с уже накопленными сведениями. Система должна отличить новое объявление от просмотренного ранее, заметить снижение цены на знакомое объявление, помнить, по какой цене предложение уже отправлялось пользователю, и не присылать одно и то же уведомление дважды. Каждая из этих операций сводится к запросу к хранимым данным, поэтому по своей сути задача относится к управлению данными.

Файловое хранение для этих данных не подходит. С ними одновременно работают несколько процессов: фоновая задача опроса записывает объявления и найденные совпадения, задача доставки отмечает отправленные уведомления, а сервис, обслуживающий бота, читает и изменяет настройки пользователей. Без согласованного параллельного доступа эти операции мешали бы друг другу. Записи связаны в цепочку из пользователя, его отслеживаемых товаров, совпадений с объявлениями и отправленных уведомлений, и удаление товара не должно оставлять в хранилище совпадения, которые ссылаются на несуществующую запись. Доставка уведомления требует транзакции: отметка об отправке и запись в журнал уведомлений фиксируются вместе, иначе сбой между двумя шагами приведёт к потере сообщения или к его повтору. Параллельный доступ, ссылочную целостность и транзакции обеспечивает СУБД, поэтому основой системы служит реляционная база данных *PostgreSQL*.

Цель курсового проекта – разработка автоматизированной информационной системы отслеживания цен на товары в виде *Telegram*-бота, который

ведёт мониторинг объявлений *Kufar.by* и уведомляет пользователей о предложениях ниже заданной цены.

Для достижения цели поставлены следующие задачи:

- проанализировать предметную область и существующие средства мониторинга объявлений;
- спроектировать базу данных и архитектуру системы;
- реализовать базу данных, периодический опрос площадки, отбор подходящих объявлений и доставку уведомлений;
- подготовить руководство для пользователя и администратора.

1 ОБЗОР ИССЛЕДУЕМОЙ ОБЛАСТИ

1.1 Описание предметной области

Значительная часть сделок с подержанными вещами в Беларуси проходит через площадки частных объявлений. Продавец публикует описание товара и цену, покупатель находит объявление через поиск и связывается с продавцом напрямую. Сама площадка в сделке не участвует, а хранит объявления, предоставляет поиск по ним и средства связи между сторонами. Крупнейшая площадка такого рода в стране – *Kufar.by*. На ней публикуются объявления о продаже транспорта, электроники, мебели, одежды, спортивного инвентаря, а также о продаже и аренде недвижимости [1].

Объём предложения велик даже в пределах отдельной категории. По выборке, выполненной через поисковый интерфейс площадки в сентябре 2026 года, в категории «Велоспорт» одновременно опубликовано 22 966 объявлений. Из них 16 494 относятся к самим велосипедам, остальные к запчастям и аксессуарам: камерам, втулкам, переключателям. В категории «Квартиры» выставлено 23 930 объявлений о продаже и 4 442 объявления о долгосрочной аренде. Из 13 028 квартир, предложенных в Минске, частные лица разместили около 800, остальные 94 % опубликованы агентствами недвижимости. Выдача при этом не стоит на месте. Проданные товары снимаются с публикации, появляются новые объявления, продавцы меняют цены в уже опубликованных.

Со стороны спроса на площадке работают две группы пользователей. Первую составляют частные покупатели, которым нужен конкретный товар для себя: велосипед с определённым размером рамы, ноутбук с заданным объёмом оперативной памяти, квартира в выбранном районе. Покупатель ищет эпизодически, в течение нескольких дней или недель, пока не найдёт подходящее предложение. Ко второй группе относятся перекупщики, которые приобретают товар ниже рыночной цены и затем продают его дороже. Перекупщик следит за площадкой постоянно и сразу по нескольким категориям, а доход перекупщика прямо зависит от скорости обнаружения выгодного объявления. Интенсивность поиска у двух групп разная, однако потребность одна и та же: узнать о подходящем объявлении раньше других.

Рынок подержанных товаров устроен иначе, чем розничная торговля. В магазине один и тот же товар продаётся под одним артикулом, а цена меняется постепенно и примерно одинаково у разных продавцов. На площадке объявлений каждое предложение уникально. Два велосипеда одной модели различаются состоянием, комплектацией и ценой, причём цену назначает сам

продавец. При спешной продаже или плохом знании рынка цена оказывается заметно ниже обычной. Такое объявление быстро находит покупателя, и выгодное предложение нередко живёт на площадке лишь несколько часов. Пользователь, который проверяет выдачу утром и вечером, регулярно пропускает объявления, выставленные и проданные в промежутке между проверками.

Ручной мониторинг площадки упирается в следующие проблемы:

- объявления появляются круглосуточно, в том числе ночью, и человек не может постоянно следить за выдачей;
- человек не удерживает в памяти рыночный минимум по каждому интересующему товару и не всегда может сразу оценить, выгодна ли увиденная цена;
- при повторном просмотре большую часть выдачи, до 90 %, составляют уже виденные объявления, и среди них приходится заново искать несколько новых;
- в карточке объявления отображается только текущая цена, поэтому снижение цены на просмотренное ранее предложение легко пропустить;
- сравнение объявлений сразу по нескольким параметрам (цене, состоянию, характеристикам товара, району) утомительно и сопровождается ошибками.

Частично эти проблемы закрывает встроенная функция площадки: сохранённые поиски. Пользователь настраивает фильтры выдачи, сохраняет поиск в личном кабинете и получает оповещения о новых объявлениях, которые под него попадают. Возможности механизма ограничены. Цена в сохранённом поиске задаётся лишь как фильтр выдачи, снижение цены на объявление, опубликованное раньше, отдельным событием не считается, история цен не ведётся. Оповещения приходят только через сервисы самой площадки. Сторонняя программа прочитать сохранённые поиски не может: программный интерфейс площадки отвечает на такой запрос ошибкой 401, если в нём нет авторизованной сессии владельца. Подробнее этот механизм разобран в подразделе 1.2.

Если отвлечься от пользовательского интерфейса, задачу мониторинга можно описать набором объектов и запросом к ним. В предметной области выделяются следующие объекты:

- пользователь – человек, который взаимодействует с системой через *Telegram* и определяет, что именно нужно отслеживать;
- отслеживаемый товар – набор критериев одного пользователя: категория, название, фильтры по характеристикам, пороговая цена и слова-исключения;

- объявление – сведения о предложении, полученные с площадки: заголовок, описание, цена, ссылка, местоположение и дата публикации;
- совпадение – факт того, что конкретное объявление удовлетворило критериям конкретного отслеживаемого товара, вместе с типом события (новое объявление или снижение цены);
- уведомление – факт отправки пользователю сообщения о совпадении с ценой объявления на момент отправки.

К этим объектам добавляется справочник характеристик категорий площадки. Площадка не публикует перечень своих фильтров, и для построения меню выбора бренда, размера рамы или числа комнат система собирает этот справочник сама по действующим объявлениям.

Каждый цикл опроса сводится к одному и тому же запросу. Для каждого активного отслеживаемого товара нужно найти объявления, которые одновременно:

- имеют цену строго ниже пороговой;
- соответствуют категории и фильтрам товара и не содержат слов-исключений;
- опубликованы после того, как товар был добавлен в систему, либо подешевели с момента предыдущего опроса;
- ещё не отправлялись этому пользователю по такой же или более низкой цене.

Первые два условия проверяются по свежему ответу площадки. Третье и четвёртое сравнивают этот ответ с тем, что система сохранила раньше: без хранимой истории объявлений и отправленных уведомлений их проверить нельзя. Объявления, прошедшие отбор, фиксируются как совпадения и ставятся в очередь на доставку. При интервале опроса в 3 минуты система выполняет 480 циклов в сутки. В каждом цикле для каждого товара приходит до 42 объявлений, и каждое из них сверяется с базой данных. Устройство системы определяет этот поток сравнений, а отображение сообщений в нём вторично.

1.2 Обзор аналогов

Задачу отслеживания выгодных предложений уже решают несколько типов программных средств, но каждое из них рассчитано на свои условия. Для обзора выбраны три аналога, которые подходят к задаче с разных сторон: встроенный механизм самой площадки *Kufar.by*, зарубежные сервисы отслеживания цен на новые товары и открытые *Telegram*-боты для мониторинга объявлений. Каждый аналог рассмотрен по одной схеме: назначение, прин-

цип работы, достоинства и недостатки применительно к рынку подержанных товаров.

1.2.1 Сохранённые поиски *Kufar.by*

Сохранённый поиск – встроенная функция площадки *Kufar.by* для зарегистрированных пользователей. Пользователь выбирает на сайте категорию, задаёт фильтры (бренд, характеристики товара, регион, диапазон цены) и сохраняет получившийся поиск в личном кабинете. После этого площадка оповещает владельца поиска о новых объявлениях, которые попадают под заданные условия [1].

Технически сохранённый поиск сводится к набору параметров обычной поисковой выдачи. При открытии сохранённого поиска площадка переходит по ссылке, в которой закодированы все выбранные фильтры: код категории, коды значений характеристик и границы цены. Эти же коды площадка использует во внутреннем поисковом интерфейсе. Ссылка полностью воспроизводима, поэтому набор фильтров, собранный на сайте, можно перенести в другую систему, если та понимает тот же формат параметров.

Достоинства сохранённых поисков:

- функция встроена в площадку, бесплатна и не требует сторонних сервисов;
- доступен полный набор фильтров площадки по всем категориям;
- сведения о новых объявлениях поступают из первоисточника.

Недостатки проявляются, как только задача выходит за рамки «показать новые объявления»:

- цена работает только как фильтр выдачи, а не как порог выгоды, относительно которого оценивается предложение;
- система реагирует лишь на появление новых объявлений, снижение цены на уже опубликованное объявление не отслеживается;
- история цен не сохраняется, и пользователь не видит, как менялась цена объявления;
- оповещения приходят только через сервисы площадки;
- программный доступ к сохранённым поискам закрыт: запрос к ним без авторизованной сессии владельца завершается ошибкой 401, и прочитать их другой программой нельзя.

1.2.2 Сервисы отслеживания цен *CamelCamelCamel* и *Keera*

CamelCamelCamel и *Keera* – наиболее известные сервисы отслеживания цен на товары интернет-магазина *Amazon*. Пользователь указывает товар

ссылкой на страницу товара или артикулом, после чего сервис регулярно фиксирует цену и строит график изменения цены за всё время наблюдения. Пользователь задаёт желаемую цену и получает уведомление, когда фактическая цена опускается до неё. *Keera*, кроме веб-сайта, распространяется как расширение для браузера и встраивает график истории цен прямо в страницу товара. Часть возможностей *Keera* доступна только по платной подписке [2, 3].

По модели данных эти сервисы ближе всего к разрабатываемой системе. Оба сервиса хранят историю цен, сравнивают текущую цену с порогом пользователя и уведомляют о снижении цены, то есть решают задачу управления данными о ценах, а не только показа этих данных.

Достоинства сервисов:

- полная история цены с момента начала наблюдения и наглядные графики;
- пороговая цена как явное условие уведомления;
- уведомления о снижении цены до заданного уровня.

Недостатки с точки зрения рынка подержанных товаров:

- 1 Поддерживается только *Amazon*, белорусские площадки объявлений не охвачены.
- 2 Модель данных опирается на артикул: один товар имеет одну цену, меняющуюся во времени. На площадке объявлений каждое предложение уникально, артикула у него нет, и история цены по артикулу к нему неприменима.
- 3 Сервисы не ищут новые предложения по критериям, а следят только за заранее указанным товаром.
- 4 Характеристики, важные для подержанного товара (состояние, комплектация, район), не учитываются.

1.2.3 Открытые *Telegram*-боты мониторинга объявлений

На *GitHub* опубликованы небольшие проекты с открытым исходным кодом, которые отслеживают объявления на *Kufar.by*, *Avito* и *OLX* и присылают их в *Telegram*. Типичная реализация устроена просто. Пользователь передаёт боту поисковую фразу или ссылку на выдачу. Бот с заданным интервалом запрашивает выдачу площадки, сравнивает идентификаторы полученных объявлений со списком уже виденных и о новых отправляет сообщение. Список виденных объявлений обычно хранится в файле или в памяти процесса.

Достоинства открытых ботов:

- уведомления приходят в *Telegram*, которым пользователь и так пользуется каждый день;

- боты бесплатны, а исходный код доступен для изучения и доработки;
- развернуть такого бота можно за несколько минут.

Недостатки:

1 Поиск ведётся по тексту, характеристики категорий не учитываются. Объявление с названием бренда, написанным кириллицей, по латинскому названию не находится, а по слову «велосипед» в выдачу попадают камеры и втулки.

2 Сравниваются только идентификаторы объявлений, поэтому снижение цены на уже виденное объявление остаётся незамеченным.

3 Частота запросов к площадке не ограничивается, и на отказ площадки (ответы 403 и 429) бот не реагирует, продолжая запросы в прежнем темпе до блокировки.

4 Сообщение отправляется один раз. При отказе *Telegram* уведомление теряется.

5 Хранение в файле рассчитано на одного пользователя и не выдерживает одновременной работы нескольких процессов.

1.2.4 Сравнение аналогов

Результаты обзора сведены в таблицу 1.1. В последнем столбце указаны возможности разрабатываемой системы.

Ни один из аналогов не закрывает задачу целиком. Сохранённые поиски площадки работают с полным набором её фильтров, но не отслеживают цену объявления во времени и закрыты для других программ. *CamelCamelCamel* и *Keera* хранят историю цен и работают с порогом, но рассчитаны на новые товары с артикулом и на *Amazon*. Открытые боты доставляют уведомления в *Telegram*, однако ограничиваются текстовым поиском по идентификаторам и теряют сообщения при сбоях. Свободной остаётся ниша системы, которая объединяет фильтры площадки, пороговую цену, отслеживание снижения цены и надёжную доставку уведомлений в *Telegram* для нескольких пользователей одновременно. Все перечисленные функции опираются на хранимые данные, и их реализация требует базы данных, а не файла со списком виденных объявлений.

1.3 Постановка задачи

Требуется разработать автоматизированную информационную систему отслеживания цен на товары. Пользователь взаимодействует с ней через *Telegram*-бота: задаёт критерии товара и пороговую цену, а система опрашивает площадку *Kufar.by*, отбирает подходящие объявления и присылает

Таблица 1.1 – Сравнение аналогов и разрабатываемой системы

Критерий	Сохранённые поиски <i>Kufar.by</i>	<i>CamelCamel-Camel, Keepa</i>	Открытые <i>Telegram</i> -боты	Разрабатываемая система
Работа с <i>Kufar.by</i>	+	–	+	+
Пороговая цена как условие уведомления	частично	+	частично	+
Уведомление о снижении цены	–	+	–	+
Хранение цен по объявлению	–	+	–	частично
Отбор по характеристикам категории	+	–	–	+
Уведомления в <i>Telegram</i>	–	–	+	+
Экспорт данных и отчёты	–	частично	–	+

уведомления о новых предложениях и о снижении цены. Все данные системы хранятся в реляционной базе данных под управлением *PostgreSQL*. В системе предусмотрены две роли. Пользователь отслеживает товары и получает уведомления. Администратор, кроме того, располагает повышенным лимитом отслеживаемых товаров, получает отчёт о состоянии системы и служебные оповещения о сбоях. Администраторы определяются по списку идентификаторов *Telegram*, заданному в конфигурации системы.

1.3.1 Функциональные требования

Система хранит сведения о пользователях, их отслеживаемых товарах, объявлениях, полученных с площадки, найденных совпадениях и отправленных уведомлениях, а также справочник характеристик категорий площадки. Пользователь регистрируется автоматически при первом обращении к боту, язык интерфейса (русский или английский) определяется по настройкам *Telegram* и может быть изменён.

Добавление отслеживаемого товара выполняется пошаговым диалогом. Пользователь выбирает категорию, вводит название, выбирает значения харак-

теристик, задаёт пороговую цену и при желании слова-исключения, после чего подтверждает введённые данные. Для категорий «Велосипеды», «Ноутбуки» и «Квартиры (долгосрочная аренда)» отбор ведётся по характеристикам самой площадки: бренду, размеру рамы, объёму памяти, числу комнат, району и другим. В этих категориях у фильтра можно выбрать несколько значений, числовые характеристики (площадь, этаж, год постройки) вводятся диапазоном, а название товара не обязательно. Кроме ручного ввода, пользователь может передать боту ссылку на поиск *Kufar.by*: система переносит из неё категорию, фильтры и цену и сообщает, какие параметры ссылки не поддерживаются. Сразу после добавления товара пользователь получает до пяти самых дешёвых подходящих предложений, уже опубликованных на площадке.

Пользователь просматривает список своих товаров, открывает карточку товара, изменяет его название, цену, слова и фильтры, приостанавливает и возобновляет отслеживание, удаляет товар. Список товаров выгружается в файл формата *CSV* и загружается обратно, причём каждая строка файла проверяется так же, как при ручном вводе.

Система опрашивает площадку каждые 3 минуты и отправляет уведомление в двух случаях: объявление опубликовано после добавления товара и стоит дешевле порога, либо уже известное объявление подешевело. Уведомление содержит фотографии, цену, описание и ссылку на объявление, а также кнопки оценки «Интересно», «Не то» и «Куплю». Одно и то же объявление повторно приходит пользователю только по более низкой цене.

По запросу формируется отчёт в формате *Microsoft Excel* о предложениях, найденных за 7, 30 или 90 дней, с выгодой относительно порога, статусом доставки и оценкой пользователя. Администратору дополнительно доступен отчёт о состоянии системы. Пользователь может привязать адрес электронной почты с подтверждением кодом и получать на него сводные копии уведомлений и отчёты.

Число отслеживаемых товаров ограничено: 3 для пользователя, 10 для администратора и не более 25 для всей системы. Достигнув предела, пользователь получает объяснение, какое именно ограничение сработало.

1.3.2 Удобство использования

Отдельного клиентского приложения не требуется: пользователь работает через любой клиент *Telegram* на смартфоне (*Android*, *iOS*), компьютере или в браузере. Управление построено на командах и кнопках под сообщениями, ввод текста нужен только для названия, цены, диапазонов и слов-исключений. Длинные списки значений (бренды, процессоры) разбиваются на страницы

по 8 значений. Диалог на любом шаге можно отменить. На каждое действие бот отвечает сообщением: если данные введены неверно, сервис временно недоступен или диалог был прерван, пользователь видит понятное объяснение на своём языке, а не кнопку, которая перестала реагировать.

1.3.3 Надёжность

Система должна работать круглосуточно. Каждый компонент выполняется в отдельном контейнере, который при сбое перезапускается автоматически, а состояние контейнеров проверяется регулярными проверками работоспособности. Зависание бота обнаруживается по отсутствию периодической отметки о его активности.

Найденное совпадение сначала сохраняется в базе данных, и только затем отдельная задача отправляет уведомление. Если *Telegram* не принял сообщение, совпадение остаётся в очереди и отправляется повторно, всего до 5 попыток. Уведомления старше 3 суток считаются неактуальными и не отправляются. Сбой почтового сервера не должен задерживать отправку в *Telegram*, поэтому копии уведомлений на электронную почту ведут отдельную очередь.

При отказе площадки обслуживать запросы (ответы с кодами 403 и 429) цикл опроса прерывается, а следующие циклы пропускаются с нарастающей паузой: 1, 2, 4, 8 и 16 циклов. Администратор получает об этом одно оповещение, повторные сообщения на каждом цикле не отправляются. Об остановке, падении или потере работоспособности любого контейнера администратор получает отдельное оповещение.

База данных копируется ежедневно, хранятся 7 последних резервных копий, пригодность копии проверяется восстановлением во временную базу. Структура базы данных изменяется только через версионированные миграции, которые применяются автоматически при запуске системы. Устаревшие данные удаляются по расписанию: объявления без совпадений через 30 дней, записи журнала уведомлений через 90 дней.

1.3.4 Производительность

Интервал опроса площадки составляет 3 минуты, к площадке отправляется не более 30 запросов в минуту с паузой между запросами. Каждый отслеживаемый товар требует одного запроса за цикл, и ограничение в 25 товаров на систему гарантирует, что цикл завершается до начала следующего. Если цикл занимает более 80 % интервала, система предупреждает об этом в журнале работы. Очередь уведомлений обрабатывается каждую минуту,

поэтому от публикации объявления до получения уведомления проходит не более нескольких минут. Чтобы не перегружать пользователя, по одному товару отправляется не более 5 уведомлений в час, а одному пользователю не более 30 в сутки, причём первыми уходят самые дешёвые предложения. Поиск объявления по паре «площадка – идентификатор объявления» выполняется на каждом цикле для каждого полученного объявления и обеспечивается составным индексом.

1.3.5 Безопасность

Доступ к базе данных имеет только серверный сервис; бот получает и изменяет данные исключительно через его программный интерфейс. Каждый запрос к интерфейсу сопровождается общим секретным ключом, при несовпадении ключа запрос отклоняется, а если ключ не задан, сервис не обслуживает запросы вовсе. Число запросов с одного адреса ограничено 300 в минуту, адрес блокируется на 15 минут после 10 неудачных попыток ввода ключа за 5 минут. Документация программного интерфейса закрыта тем же ключом.

Пользователь видит и изменяет только собственные товары. Код подтверждения адреса электронной почты хранится в виде *HMAC*-хеша, действует 15 минут, допускает 5 попыток ввода и отправляется повторно не чаще раза в минуту. Без подтверждения на адрес ничего не отправляется, поэтому использовать систему для рассылки писем на чужой адрес нельзя. Значения из загружаемых и выгружаемых файлов *CSV*, похожие на формулы, экранируются, а текст объявлений экранируется перед вставкой в сообщение. Секретные параметры (токен бота, пароль базы данных, ключ интерфейса) хранятся в файле окружения вне исходного кода. Контейнеры работают от имени непривилегированного пользователя, а служба мониторинга получает события *Docker* только на чтение через отдельный посредник.

1.3.6 Аппаратное и программное обеспечение для клиента и сервера

От клиента требуется только устройство с установленным клиентом *Telegram* или браузером с доступом к веб-версии *Telegram* и подключение к сети Интернет. Для получения копий уведомлений на электронную почту подходит любой почтовый клиент.

Серверная часть рассчитана на один сервер под управлением *Linux* с установленными *Docker* и *Docker Compose*. Рекомендуемая конфигурация сервера: 64-битный процессор с двумя ядрами, не менее 2 ГБ оперативной памяти и не менее 20 ГБ дискового пространства на *SSD* для базы данных, резервных копий и журналов работы. Программная часть реализуется на языке

Python 3.11 и включает СУБД *PostgreSQL 16*, веб-фреймворк *FastAPI* для серверного интерфейса, библиотеку *aiogram 3* для работы с *Telegram Bot API*, асинхронный драйвер *asyncpg*, через который выполняются SQL-запросы к базе данных, *Alembic* для версионирования схемы с миграциями в виде SQL-скриптов и *APScheduler* для периодических задач. Для работы системы нужен стабильный доступ к *Kufar.by* и к серверам *Telegram*.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Проектирование баз данных

Проектирование базы данных выполнено в два этапа: концептуальное и логическое. На концептуальном этапе построена *ER*-модель в нотации Питера Чена. Модель описывает предметную область в терминах сущностей, их атрибутов и связей и не зависит от конкретной СУБД. На логическом этапе модель переведена в набор реляционных таблиц с первичными и внешними ключами [4]. Физическая реализация схемы в *PostgreSQL* рассмотрена в подразделе 3.2.

Информационное пространство системы делится на три группы данных. Первая описывает пользователей и то, что они отслеживают. Вторая накапливается при работе системы: объявления с площадки, найденные совпадения и журнал отправленных уведомлений. К третьей относится справочник характеристик категорий площадки, который система собирает сама и по которому строит меню фильтров. На концептуальном уровне все три группы входят в одну модель, поскольку связаны между собой через отслеживаемый товар.

Центральная сущность модели – «Пользователь». С ней связана сущность «Отслеживаемый товар», которая хранит критерии поиска. Сущность «Объявление» содержит сведения, полученные с площадки, и существует независимо от пользователей: одно объявление может подойти под товары нескольких людей. Связь товара и объявления вынесена в отдельную сущность «Совпадение», поскольку у самой связи есть собственные атрибуты: тип события, показатель выгоды, состояние доставки. Факт отправки сообщения фиксирует сущность «Уведомление». Справочную часть составляют сущности «Атрибут фильтра» и «Значение атрибута». *ER*-диаграмма приведена на рисунке 2.1. На концептуальном уровне сущности, связи и атрибуты названы по-русски и от имён таблиц базы данных не зависят; соответствие имён приведено ниже, при описании логической схемы.

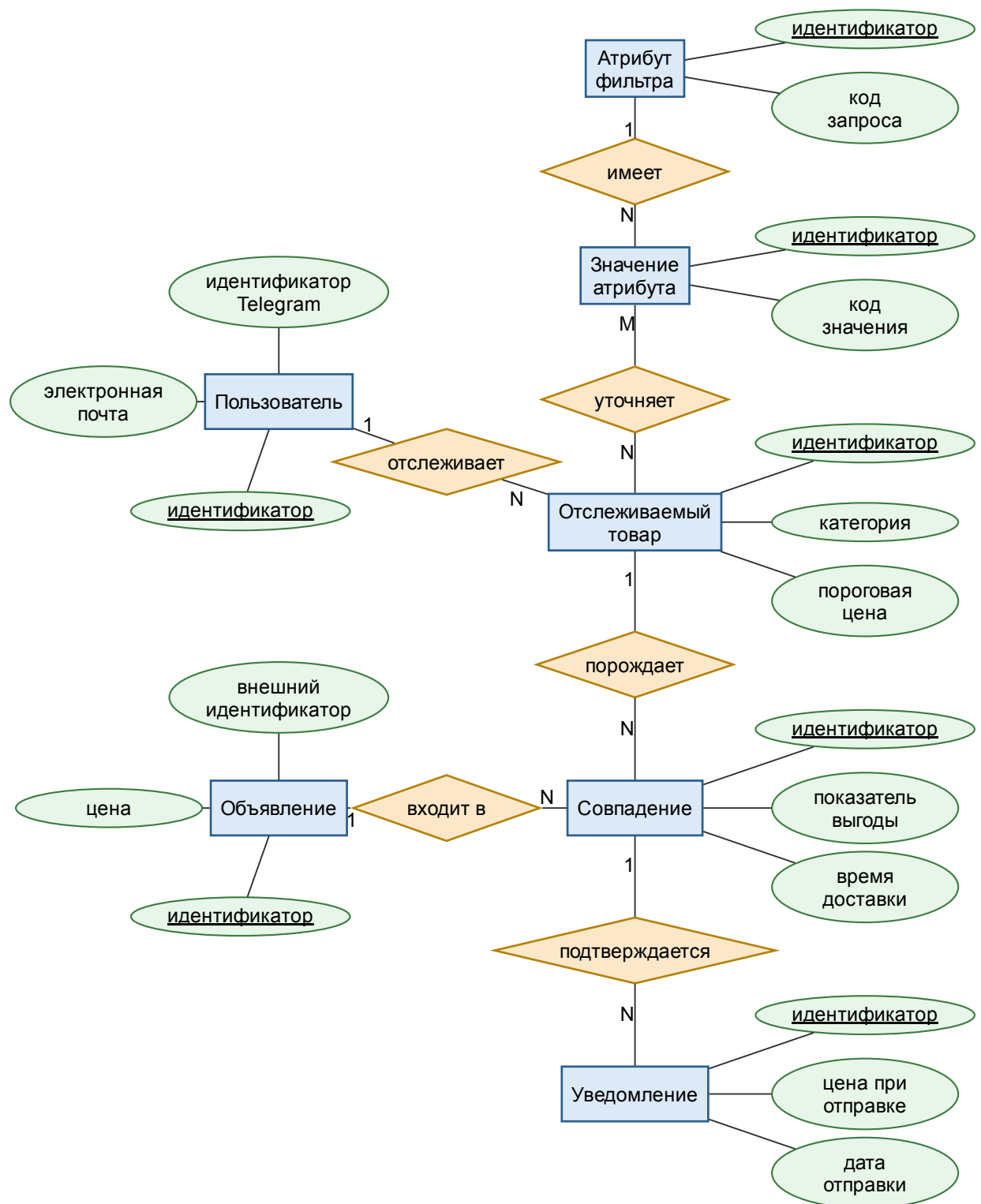


Рисунок 2.1 – ER-диаграмма базы данных в нотации Питера
Цена

Сущность «Пользователь»

– идентификатор – первичный ключ, внутренний номер пользователя в системе;

- идентификатор *Telegram* – номер учётной записи, по которому бот узнаёт пользователя; должен быть уникальным;
- имя учётной записи – публичное имя в *Telegram* (может отсутствовать);
- имя из профиля – имя, которое бот использует в обращениях;
- язык интерфейса – язык меню и уведомлений (русский или английский);
- признак активности – снимается, если пользователь заблокировал бота, чтобы система не пыталась отправлять ему сообщения;
- признак уведомлений – включена ли отправка уведомлений в *Telegram*;
- электронная почта – адрес для копий уведомлений и отчётов (может отсутствовать);
- признак подтверждения почты – подтверждён ли адрес кодом; без подтверждения письма на адрес не отправляются;
- признак почтовых копий – отправлять ли копии уведомлений на подтверждённый адрес;
- хеш кода подтверждения – *НМАС*-свёртка шестизначного кода; сам код в базе данных не хранится;
- срок действия кода – момент, после которого код подтверждения становится недействительным;
- время отправки кода – момент последней отправки; ограничивает частоту повторных запросов;
- число попыток ввода кода – после пятой неудачной попытки код нужно запросить заново;
- дата регистрации – момент первого обращения пользователя к боту;
- дата изменения – момент последнего изменения записи.

Сущность «Отслеживаемый товар»

- идентификатор – первичный ключ, уникальный номер товара;
- владелец – ссылка на пользователя, который добавил товар;
- название – название товара, по которому ведётся текстовый отбор; в категориях с фильтрами площадки может быть пустым;
- категория – одна из категорий фиксированного перечня (велосипеды, ноутбуки, аренда квартир, электроника, инструменты и другие);
- пороговая цена – цена в белорусских рублях, ниже которой объявление считается выгодным;
- слова поиска – список слов, каждое из которых должно встретиться в заголовке или описании объявления;
- слова-исключения – список слов, при наличии которых объявление

отклоняется;

- фильтры площадки – набор кодов характеристик и выбранных значений (бренд, размер рамы, число комнат, диапазон площади), которые передаются площадке в поисковом запросе;

- признак активности – снимается, когда пользователь приостанавливает отслеживание;

- дата добавления – момент добавления товара; объявления, опубликованные позже этого момента, считаются новыми.

Сущность «Объявление»

- идентификатор – первичный ключ, внутренний номер объявления;

- площадка – источник объявления; в текущей версии системы это *Kufar.by*;

- внешний идентификатор – номер объявления на площадке; вместе с площадкой однозначно определяет объявление;

- заголовок – заголовок объявления, заданный продавцом;

- описание – текст объявления, а при его отсутствии перечень характеристик товара;

- цена – цена в белорусских рублях (может отсутствовать, если цена договорная);

- валюта – валюта, в которой указана цена;

- ссылка – адрес страницы объявления на площадке;

- изображения – список адресов фотографий товара;

- местоположение – регион и город или район;

- сведения о продавце – имя и номер учётной записи продавца на площадке;

- состояние – состояние товара, указанное продавцом (новое, бывшее в употреблении);

- исходный ответ площадки – полный ответ по объявлению в формате *JSON*;

- дата публикации – момент размещения объявления на площадке;

- первое появление – момент, когда система впервые получила объявление;

- последнее появление – момент последнего получения объявления; по нему удаляются устаревшие записи.

Сущность «Совпадение»

- идентификатор – первичный ключ, уникальный номер совпадения;

- отслеживаемый товар – ссылка на товар, критериям которого удовлетворяло объявление;

- объявление – ссылка на подошедшее объявление;
- показатель выгоды – доля, на которую цена объявления ниже пороговой (от нуля до единицы);
- подробности отбора – тип события (новое объявление или снижение цены), прежняя цена и порог на момент отбора;
- время доставки – момент успешной отправки уведомления в *Telegram*; пустое значение означает, что совпадение ждёт в очереди;
- число попыток – количество неудачных отправок; после пятой попытки совпадение исключается из очереди;
- текст ошибки – причина последней неудачной отправки;
- время почтовой копии – момент отправки копии уведомления на электронную почту;
- число почтовых попыток – счётчик неудачных попыток для отдельной почтовой очереди;
- оценка пользователя – оценка предложения кнопкой под уведомлением («Интересно», «Не то», «Куплю»);
- дата создания – момент, когда объявление прошло отбор.

Сущность «Уведомление»

- идентификатор – первичный ключ, уникальный номер записи журнала;
- получатель – ссылка на пользователя, которому отправлено сообщение;
- отслеживаемый товар – ссылка на товар, по которому пришло уведомление;
- объявление – ссылка на объявление из уведомления;
- совпадение – ссылка на совпадение, по которому отправлено сообщение (может отсутствовать);
- площадка и внешний идентификатор – копия идентификации объявления, по которой проверяется, получал ли пользователь это объявление раньше;
- цена при отправке – цена объявления на момент отправки; повторно объявление отправляется только по более низкой цене;
- показатель выгоды – показатель выгоды совпадения на момент отправки;
- дата отправки – используется для часового и суточного ограничений и для удаления записей старше 90 дней.

Сущность «Атрибут фильтра»

- идентификатор – первичный ключ, уникальный номер характеристики;

- категория – код категории *Kufar.by*, к которой относится характеристика;
- внутреннее имя – имя характеристики на площадке; вместе с категорией должно быть уникальным;
- код запроса – короткий код, под которым площадка принимает характеристику в поисковом запросе;
- название – название характеристики для меню бота (например, «Производитель»);
- частота – число объявлений выборки, в которых встретилась характеристика;
- дата обновления – момент последнего сбора справочника.

Сущность «Значение атрибута»

- идентификатор – первичный ключ, уникальный номер значения;
- атрибут – ссылка на характеристику, которой принадлежит значение;
- код значения – код, который передаётся площадке в запросе; вместе с атрибутом должен быть уникальным;
- название значения – название для кнопки меню (например, «Trek»);
- родительский регион – код региона для значений «город» и «район» (может отсутствовать);
- частота – число объявлений выборки с этим значением; определяет порядок значений в меню;
- дата обновления – момент последнего сбора справочника.

Связи между сущностями имеют разную мощность. Сущности «Пользователь» и «Отслеживаемый товар» связаны отношением «один ко многим»: один пользователь отслеживает несколько товаров, а каждый товар принадлежит одному пользователю. Между сущностями «Отслеживаемый товар» и «Объявление» существует отношение «многие ко многим»: одно объявление может подойти под товары разных пользователей, а по одному товару находится много объявлений. Это отношение реализовано через сущность «Совпадение», которая связана с товаром и с объявлением отношениями «один ко многим». Для каждой пары товара и объявления хранится одно совпадение: при снижении цены система обновляет его, а не создаёт новое.

Сущности «Совпадение» и «Уведомление» связаны отношением «один ко многим». Если объявление подешевело после первой отправки, пользователь должен получить по тому же совпадению ещё одно уведомление с новой ценой, и в журнале появится вторая запись. В журнале уведомлений получатель хранится напрямую, чтобы часовое и суточное ограничения подсчитывались без обращения к таблицам товаров и совпадений. На концептуальной

схеме эта связь не показана: пользователь однозначно определяется через совпадение и отслеживаемый товар.

В справочной части сущности «Атрибут фильтра» и «Значение атрибута» связаны отношением «один ко многим». Сущности «Значение атрибута» и «Отслеживаемый товар» образуют отношение «многие ко многим»: товар уточняется несколькими значениями, а одно значение (например, бренд) выбрано в товарах разных пользователей. В отдельную таблицу эта связь не вынесена. Выбранные коды значений хранятся в самом товаре как набор в формате *JSON*, поскольку в таком же виде они подставляются в поисковый запрос к площадке, а справочник пересобирается каждую неделю и не должен каскадно менять товары пользователей.

Логическая схема базы данных приведена на рисунке 2.2. Каждая сущность стала таблицей с суррогатным первичным ключом *id*, а связи «один ко многим» выражены внешними ключами. Сущности «Пользователь», «Отслеживаемый товар», «Объявление», «Совпадение» и «Уведомление» соответствуют таблицам *users*, *tracked_products*, *listings*, *matches* и *sent_notifications*, справочная часть – таблицам *kufar_attributes* и *kufar_attribute_values*. Кроме таблиц, соответствующих сущностям модели, в схему входит резервная таблица пользовательских атрибутов товара *custom_attributes*. Она создана для будущего расширения системы, в текущей версии не заполняется и в концептуальную модель не включена.

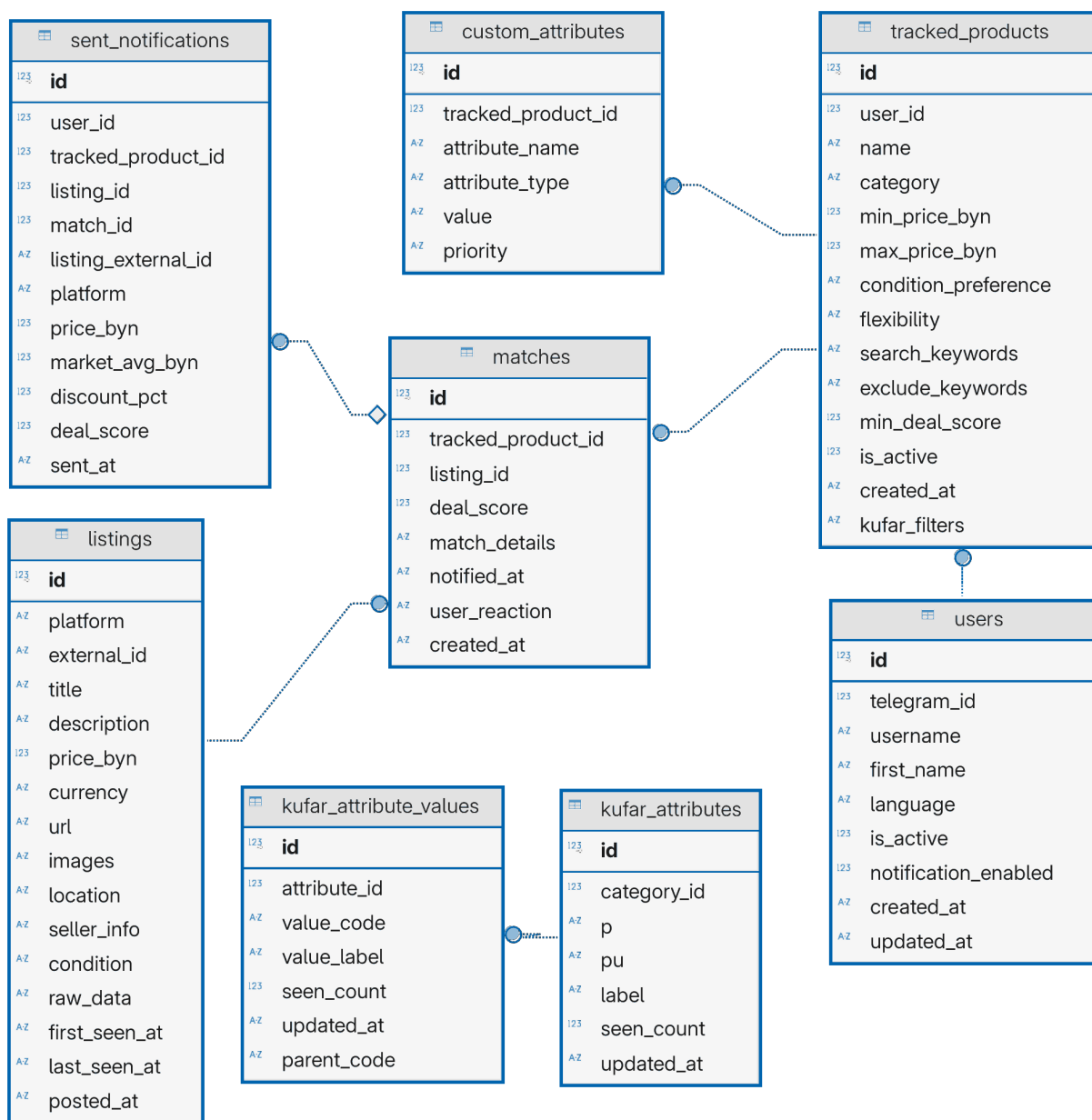


Рисунок 2.2 – Логическая схема базы данных

Разделение данных на пользовательские, рабочие и справочные определяет дальнейшую работу со схемой. Пользовательские таблицы меняются только по командам пользователя и невелики. Рабочие таблицы растут с каждым циклом опроса, поэтому для них предусмотрено удаление устаревших записей и индексы под запросы цикла. Справочные таблицы перезаписываются по расписанию и на пользовательские данные не ссылаются.

2.2 Проектирование системы

До начала реализации нужно определить границы системы: какие действия выполняет пользователь, что система делает сама по расписанию и с какими внешними сервисами обменивается данными. Эти границы описаны диаграммами вариантов использования и контекста. Поведение системы во время работы показано диаграммой последовательности и схемой алгоритма мониторинга. Диаграммы строились как рабочий материал для реализации: по ним определялось, какие данные читает и записывает каждая часть системы и в каком порядке.

2.2.1 Диаграмма вариантов использования

Диаграмма вариантов использования (*Use Case Diagram*) описывает функциональные требования к системе с точки зрения тех, кто с ней взаимодействует. На ней показано, что система делает для пользователя, без деталей реализации. Диаграмма приведена на рисунке 2.3.

при обновлении справочника фильтров;

– «Почтовый сервер» – внешняя система, через которую отправляются коды подтверждения, копии уведомлений и отчёты.

Названия вариантов использования заданы глаголами, поскольку вариант описывает действие, а не объект. Операции над отслеживаемым товаром образуют полный набор: «Добавить товар», «Просмотреть список товаров», «Просмотреть карточку товара», «Изменить товар» и «Удалить товар». Такой набор из пяти операций принято обозначать сокращением *CRUDL* по первым буквам английских названий создания, чтения, изменения, удаления и получения списка.

Варианты «Опросить площадку» и «Обновить справочник» запускаются не человеком, а планировщиком системы по расписанию. Добавление товара включает выбор фильтров категории и проверку лимита, изменение товара включает тот же выбор фильтров, а импорт фильтров из ссылки *Kufar.by* расширяет добавление как альтернативный способ задать критерии. Привязка электронной почты включает её подтверждение кодом, без которого письма на адрес не отправляются.

Модель вариантов использования разграничивает функции системы между двумя ролями. Пользователю доступна вся работа с отслеживаемыми товарами, уведомлениями, отчётами и почтой. Администратор получает эти же функции и, кроме них, средства контроля: отчёт о состоянии системы и оповещения о сбоях. Большинство вариантов использования заканчивается записью в базу данных, а получение уведомлений целиком построено на данных, накопленных предыдущими циклами опроса.

2.2.2 Диаграмма контекста

Диаграмма контекста первого уровня модели *C4* показывает систему как единое целое вместе с людьми и внешними системами, с которыми происходит обмен данными. Внутреннее устройство на этом уровне не раскрывается [5]. Диаграмма приведена на рисунке 2.4.

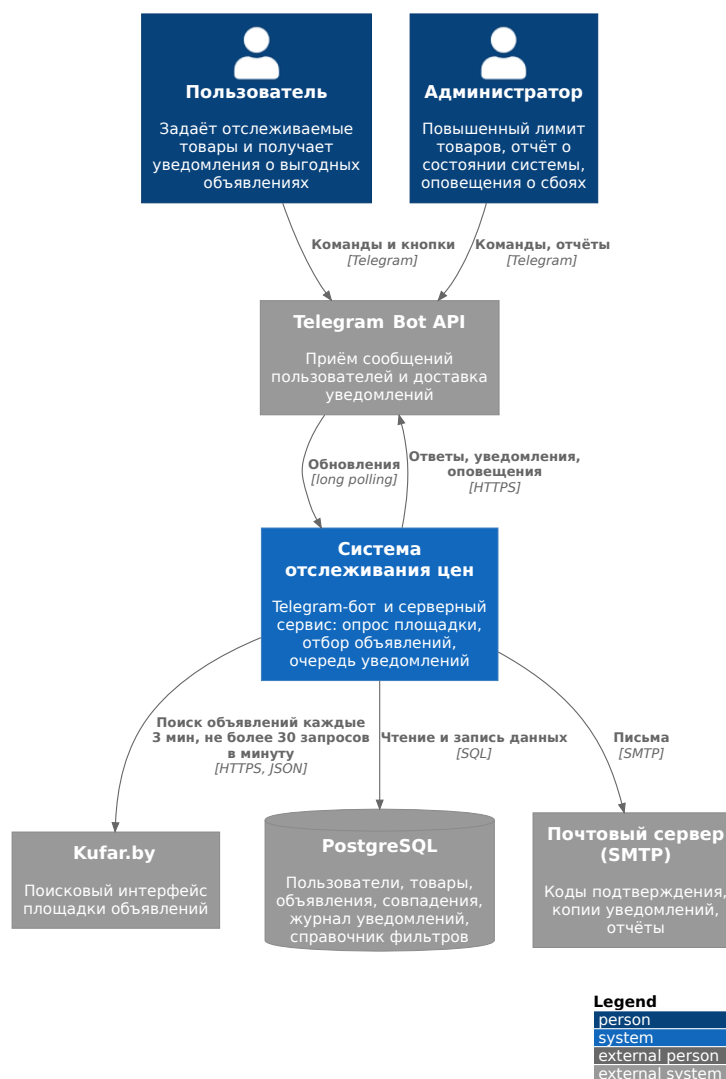


Рисунок 2.4 – Диаграмма контекста системы (C4, уровень 1)

В центре диаграммы находится система отслеживания цен. Пользователь и администратор обращаются к ней не напрямую, а через *Telegram*: команды и нажатия кнопок попадают в *Telegram Bot API*, откуда система забирает их методом длинного опроса (*long polling*). Тем же каналом система отправляет ответы, уведомления и служебные оповещения [6].

Площадка *Kufar.by* выступает источником данных. Система обращается к её поисковому интерфейсу по *HTTPS* и получает объявления в формате *JSON*. Обращения ограничены: опрос выполняется раз в 3 минуты, и за минуту отправляется не более 30 запросов.

Все данные системы хранит СУБД *PostgreSQL*. Система обращается к ней *SQL*-запросами. Почтовый сервер получает от системы письма с кодами подтверждения, копиями уведомлений и отчётами по протоколу *SMTP*.

Граница системы проходит так, что ни одна внешняя система не обраба-

ется к ней сама по себе. Система опрашивает *Telegram* и площадку по своей инициативе и в своём темпе. Это упрощает защиту: снаружи доступен только программный интерфейс для бота, закрытый секретным ключом.

2.2.3 Диаграмма последовательности

Диаграмма последовательности (*Sequence Diagram*) показывает порядок обмена сообщениями между участниками во времени. Работа системы разделена на две независимые части, которые запускает планировщик: цикл опроса площадки и доставку уведомлений. Каждая часть показана отдельной диаграммой на рисунках 2.5 и 2.6.

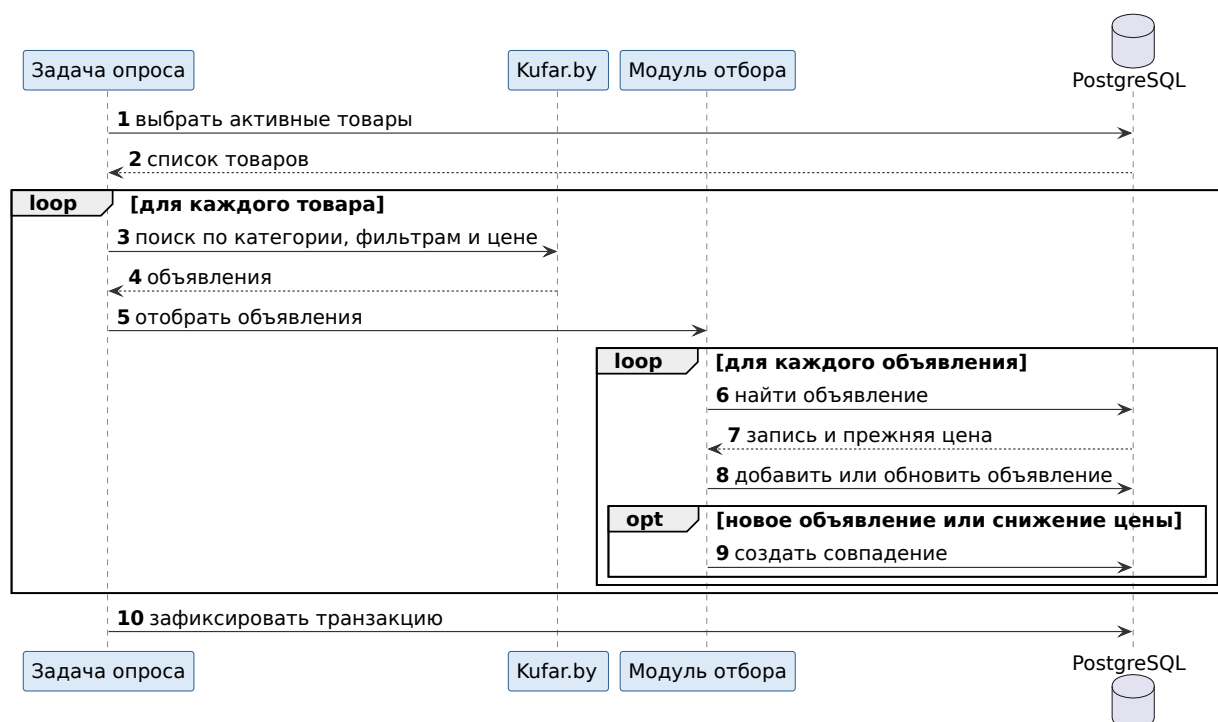


Рисунок 2.5 – Диаграмма последовательности цикла опроса площадки

Цикл опроса выполняется каждые 3 минуты. Задача опроса выбирает из базы данных активные отслеживаемые товары и для каждого отправляет площадке поисковый запрос с категорией, фильтрами и ограничением цены сверху, равным порогу товара. При отказе площадки с кодом 403 или 429 цикл прерывается сразу, не дожидаясь ответа по остальным товарам, и задача увеличивает паузу до следующего опроса.

При успешном ответе объявления передаются модулю отбора. Для каждого объявления модуль ищет в базе данных запись по паре «площадка – внешний идентификатор». Если записи нет, объявление добавляется, если

есть, обновляется, и модуль получает прежнюю цену. Затем объявление проверяется по цене, словам-исключениям и названию. Совпадение создаётся, только если объявление опубликовано после добавления товара или его цена снизилась. Изменения всего цикла фиксируются одной транзакцией.

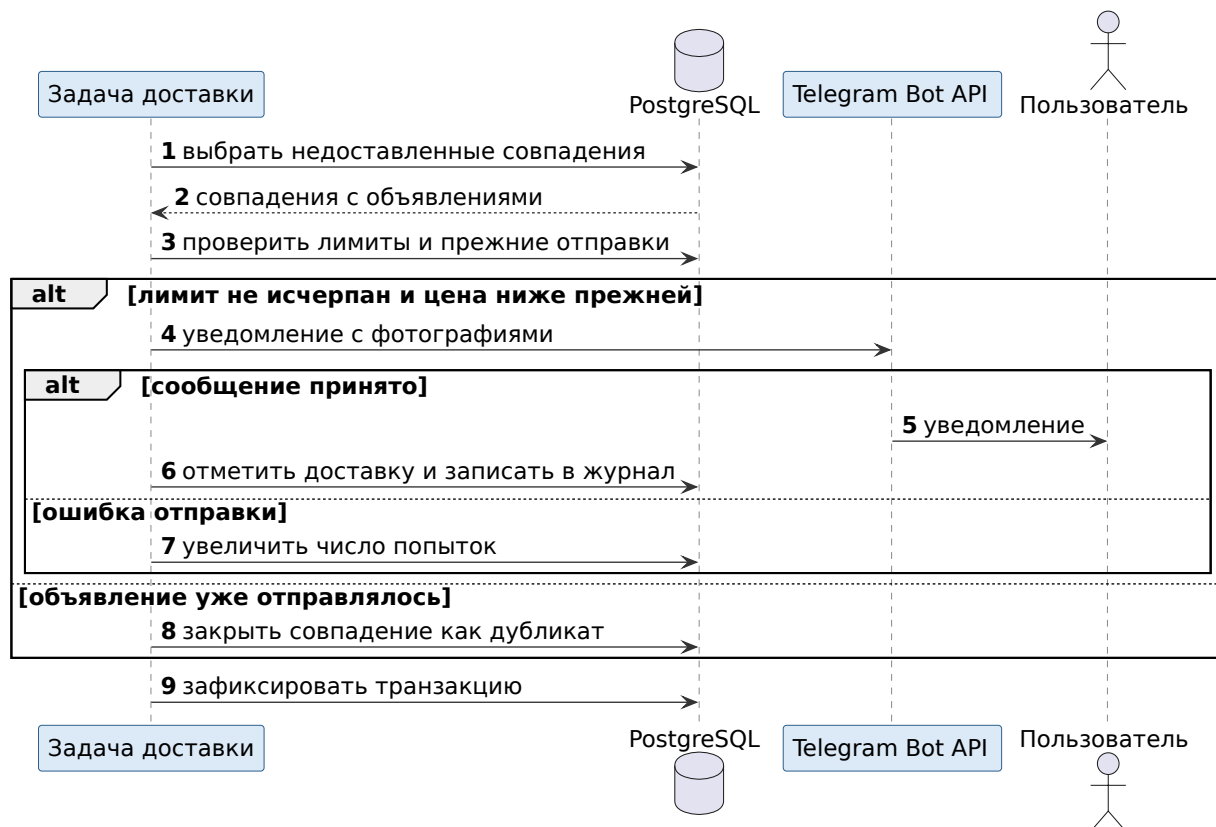


Рисунок 2.6 – Диаграмма последовательности доставки уведомлений

Доставка выполняется каждую минуту отдельной задачей. Задача выбирает недоставленные совпадения, у которых меньше пяти неудачных попыток и возраст не больше трёх суток, проверяет часовой и суточный лимиты и цену прежних отправок этого объявления. Если объявление уже приходило пользователю по такой же или меньшей цене, совпадение закрывается как дубликат. Иначе сообщение уходит через *Telegram Bot API*. При успехе совпадение отмечается доставленным и в журнал уведомлений добавляется запись, при ошибке увеличивается счётчик попыток и сохраняется текст ошибки.

Опрос и доставка разделены намеренно. Медленный ответ *Telegram* не задерживает опрос площадки, а совпадение, которое не удалось отправить, остаётся в базе данных и не теряется вместе с циклом. Очередью уведомлений в этой схеме служит обычная таблица совпадений, в которой недоставленные записи отличаются пустым временем доставки.

2.2.4 Схемы алгоритмов мониторинга и доставки уведомлений

Схемы алгоритмов детализируют логику, которая на диаграммах последовательности показана сообщениями. Схемы выполнены по ГОСТ 19.701–90: действие обозначено прямоугольником, условие – ромбом, начало и конец – прямоугольником со скруглёнными углами, ввод и вывод данных – параллелограммом. Символ «Начало» стоит вверху по центру, символ «Конец» на каждой схеме один и стоит внизу по центру. Линии потока проведены только по горизонтали и вертикали; стрелка поставлена там, где поток идёт не сверху вниз и не слева направо, то есть на возвратах к началу цикла и на боковых ветвях. Схема алгоритма цикла опроса приведена на рисунке 2.7.

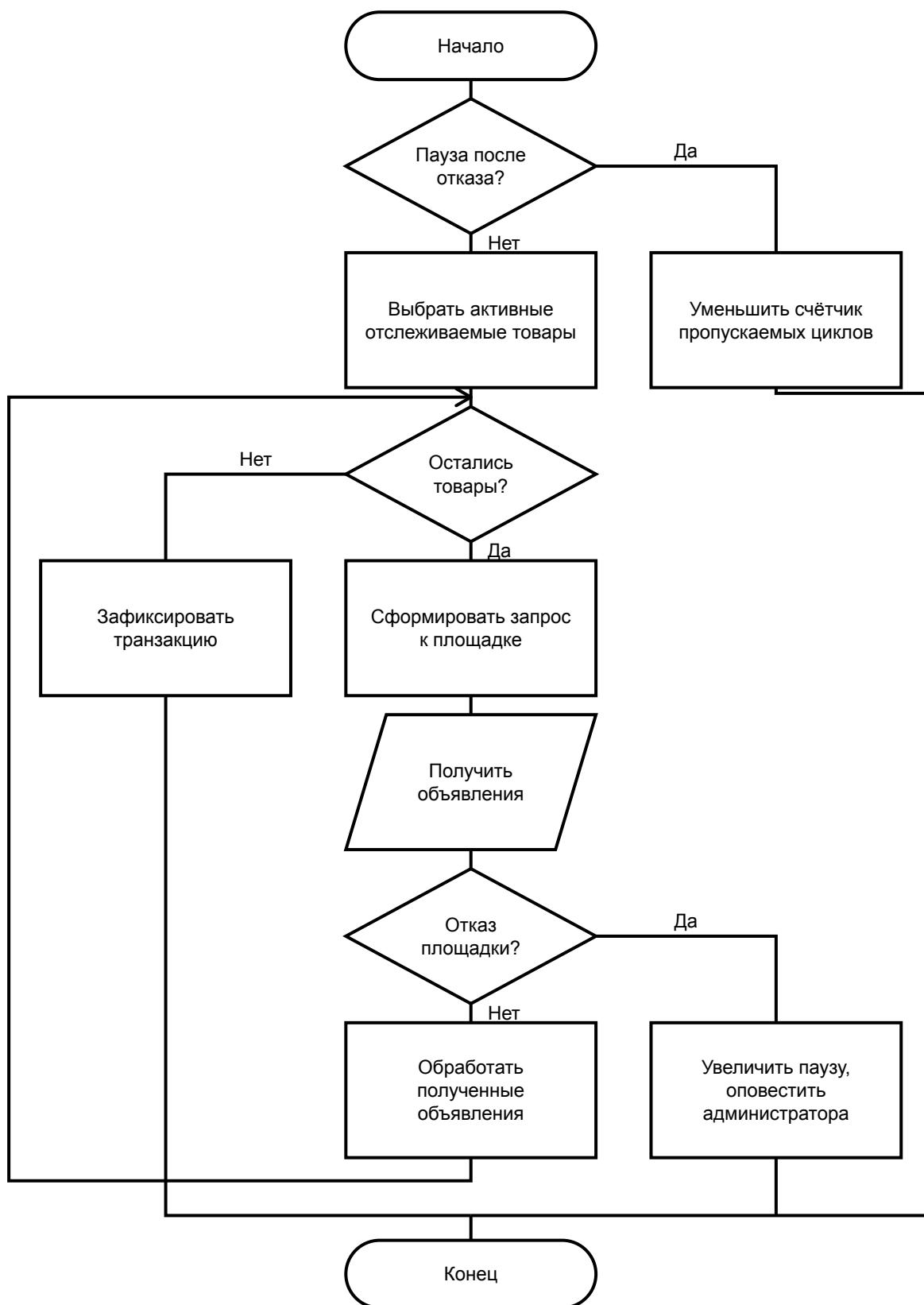


Рисунок 2.7 – Схема алгоритма цикла опроса площадки

Алгоритм начинается с проверки паузы. Если на одном из предыдущих

циклов площадка отказала в обслуживании, текущий цикл пропускается и счётчик пропускаемых циклов уменьшается на единицу. Пауза растёт при каждом отказе подряд: 1, 2, 4, 8 и не более 16 циклов. Первый успешный цикл после блокировки сбрасывает её. При интервале в 3 минуты самая длинная пауза составляет 48 минут.

Затем выбираются активные отслеживаемые товары, и для каждого формируется запрос к площадке. Товары обрабатываются строго по очереди, и нагрузка на площадку остаётся в пределах 30 запросов в минуту. Отказ площадки на любом товаре завершает весь цикл с оповещением администратора. Продолжать запросы в прежнем темпе после отказа означало бы превратить временное ограничение в длительную блокировку.

Обработка каждого полученного объявления вынесена в отдельную схему на рисунке 2.8. Объявление сначала сохраняется в базе данных, даже если потом будет отклонено: сохранённая цена нужна следующим циклам, потому что только сравнение с ней показывает снижение цены. Проверка релевантности состоит из трёх условий: цена ниже порога, в тексте нет слов-исключений, в заголовке или описании есть все слова названия товара. Прошедшее проверку объявление становится совпадением, только если оно опубликовано после добавления товара или подешевело. Объявления, которые были на площадке до добавления товара и с тех пор не подешевели, пользователь мог просмотреть сам при настройке поиска, а самые дешёвые из них получил при мгновенном поиске, поэтому о них система не сообщает.

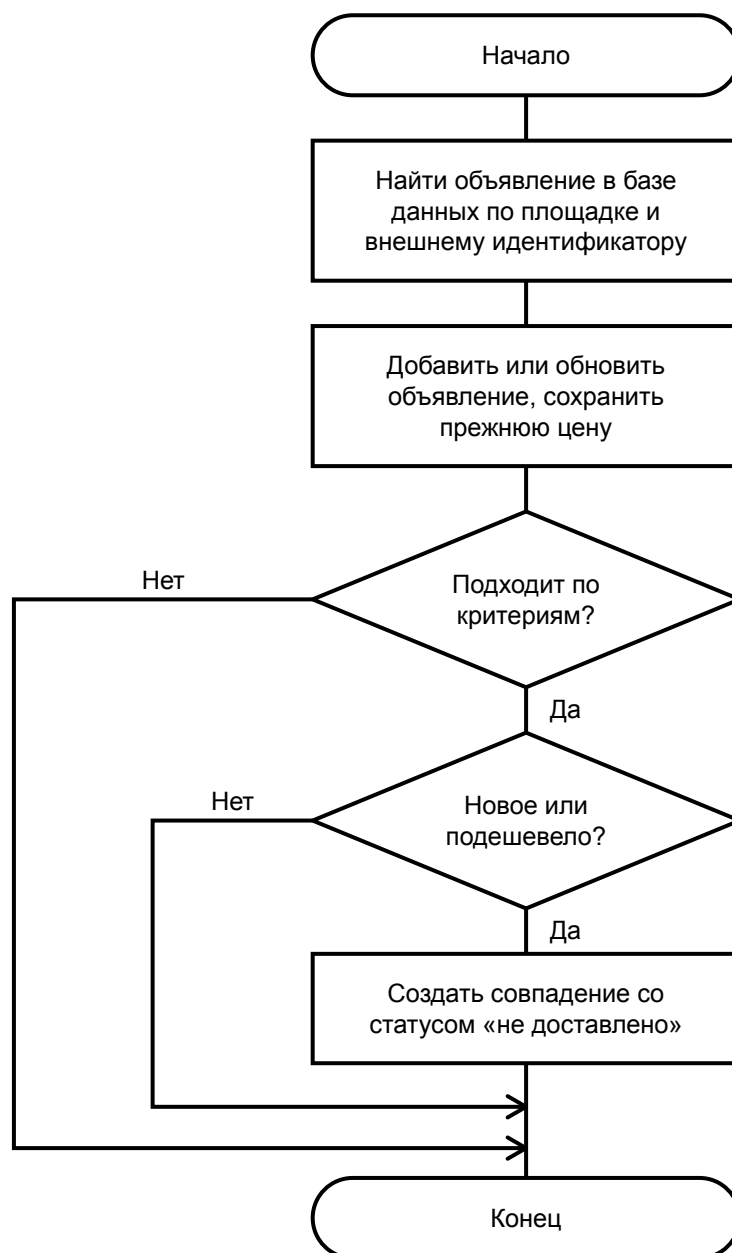


Рисунок 2.8 – Схема алгоритма обработки полученного объявления

Очередь доставки обрабатывает схема на рисунке 2.9. Для каждого совпадения проверяются три условия по порядку: не исчерпан ли лимит уведомлений, не отправлялось ли объявление по такой же или меньшей цене и принял ли *Telegram* сообщение. Первое условие оставляет совпадение в очереди до следующего часа или дня, второе закрывает его как дубликат, третье определяет, будет ли совпадение отмечено доставленным или получит ещё одну попытку. Изменения каждого запуска доставки фиксируются транзакцией, поэтому отметка о доставке и запись в журнале уведомлений сохраняются

ВМЕСТЕ.

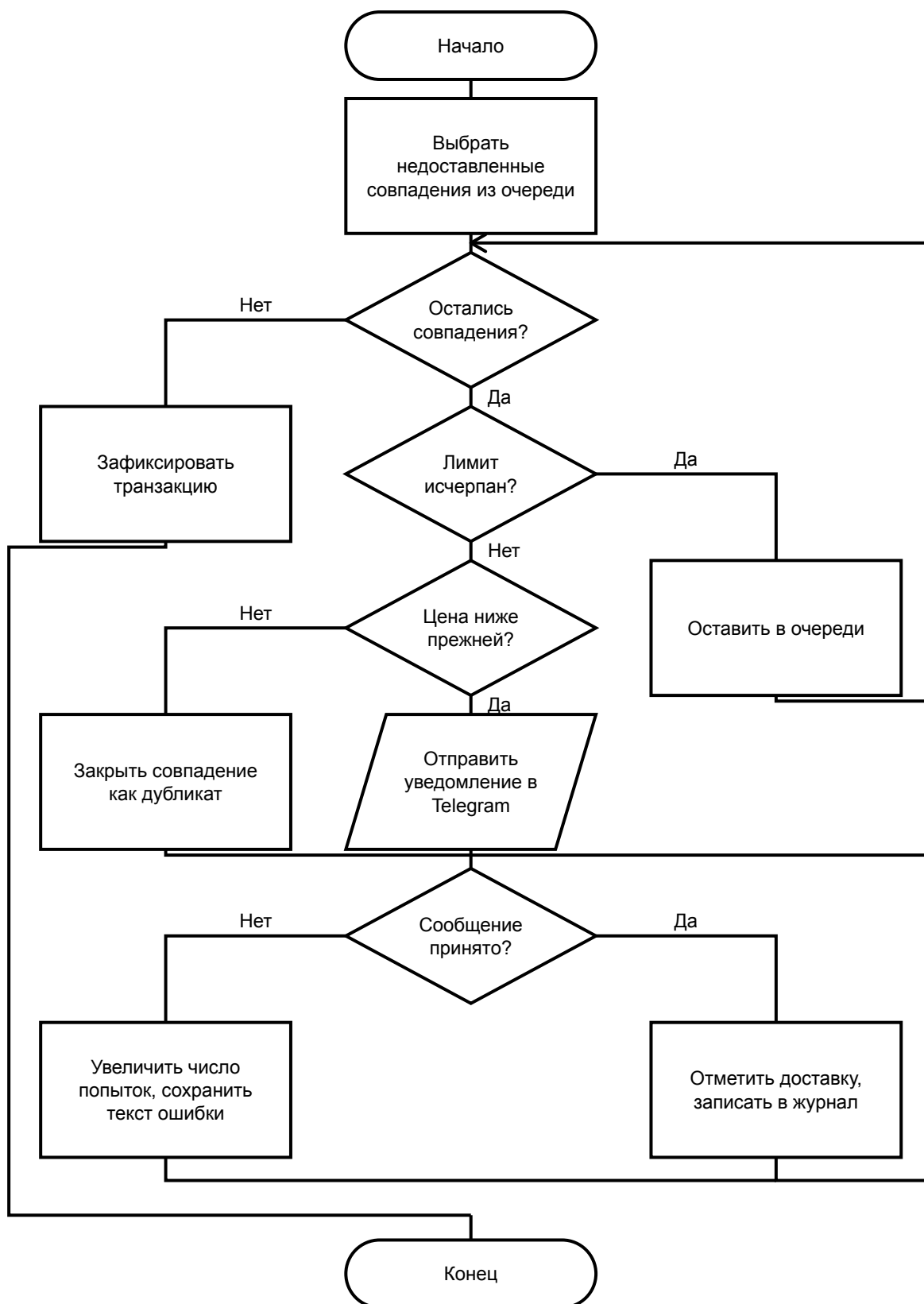


Рисунок 2.9 – Схема алгоритма доставки уведомлений

3 РАЗРАБОТКА СИСТЕМЫ

3.1 Выбор средств реализации

Средства реализации выбирались под три особенности задачи. Данные изменяются одновременно несколькими процессами. Большую часть времени система ждёт ответов от внешних сервисов: площадки, *Telegram*, почтового сервера. Система работает круглосуточно на одном сервере и должна переживать перезапуски без потери данных.

3.1.1 Язык программирования

Система написана на *Python 3.11*. В языке есть встроенная поддержка асинхронного выполнения, при котором один процесс обслуживает множество одновременных операций ввода-вывода, не создавая поток на каждую из них. Пока система ждёт ответа площадки на один запрос, параллельно обрабатываются команды пользователей и отправляются уведомления. Для *Python* существуют зрелые асинхронные библиотеки для всех внешних сервисов системы: *Telegram*, *HTTP*, *PostgreSQL*.

3.1.2 Выбор СУБД

В качестве СУБД выбрана *PostgreSQL 16*. Наряду с ней рассматривались *MySQL*, встраиваемая *SQLite* и документоориентированная *MongoDB*. Сравнение по свойствам, значимым для задачи, приведено в таблице 3.1 [7].

Таблица 3.1 – Сравнение СУБД по требованиям системы

Свойство	<i>PostgreSQL</i>	<i>MySQL</i>	<i>SQLite</i>	<i>MongoDB</i>
Работа как отдельный сетевой сервер	+	+	—	+
Одновременная запись несколькими процессами	+	+	—	+
Внешние ключи	+	+	+	—
Откат изменений схемы в транзакции	+	—	+	не требуется
Хранение <i>JSON</i> -документов	+	+	частично	+
Рекомендательные блокировки	+	+	—	—

SQLite хранит базу в одном файле и в каждый момент допускает только одну пишущую транзакцию. Первая версия бота работала на ней в одном процессе. Когда система была разделена на несколько сервисов, работающих с общими данными, от встраиваемой базы пришлось отказаться в пользу сетевой СУБД. *MongoDB* не поддерживает внешние ключи, а связи между пользователем, товаром, совпадением и уведомлением составляют основу модели данных, и их целостность пришлось бы контролировать вручную. *MySQL* подошла бы по большинству свойств, однако изменение схемы в ней не откачивается вместе с транзакцией: миграция, прерванная на середине, оставляет базу в промежуточном состоянии.

PostgreSQL закрывает все требования. Внешние ключи и транзакции обеспечивают целостность цепочки от пользователя до уведомления. Тип *JSON* хранит данные переменной структуры: фильтры площадки, у которых набор характеристик свой для каждой категории, списки слов, адреса фотографий, исходный ответ площадки. Выделять под них отдельные таблицы не нужно. Рекомендательная блокировка (*advisory lock*) защищает от одновременного применения миграций несколькими экземплярами сервиса при запуске.

3.1.3 Доступ к базе данных и миграции

Запросы к базе данных записываются на SQL и выполняются через асинхронный драйвер *asyncpg*, который работает с сетевым протоколом *PostgreSQL* напрямую. Структура базы данных описывается версионированными миграциями под управлением *Alembic*. Каждая миграция описывается SQL-скриптом перехода между двумя версиями схемы и содержит обратный переход. Миграции применяются автоматически при запуске серверного сервиса, поэтому схема рабочей базы всегда соответствует версии кода [8, 9].

3.1.4 Серверный интерфейс

Программный интерфейс, через который бот получает и изменяет данные, построен на веб-фреймворке *FastAPI* и запускается сервером *uvicorn*. *FastAPI* асинхронный и проверяет входные данные по объявленным схемам *pydantic*: запрос с отрицательной ценой или слишком длинным названием отклоняется до обращения к базе данных. Описание интерфейса формируется автоматически по коду. *Flask* требует для тех же возможностей набора расширений, а *Django* рассчитан на веб-приложения с шаблонами страниц и административной панелью, которые системе не нужны [10].

3.1.5 Работа с *Telegram*

Бот построен на библиотеке *aiogram* 3. Библиотека асинхронная и поддерживает конечные автоматы для многошаговых диалогов: добавление товара проходит до восьми шагов, и на каждом шаге сохраняется состояние диалога пользователя. Обработчики разделяются по маршрутизаторам, а общая логика, например определение пользователя по каждому входящему сообщению, выносится в промежуточный слой [11].

3.1.6 Периодические задачи

Опрос площадки, доставка уведомлений, рассылка почтовых копий, очистка устаревших данных и обновление справочника фильтров запускаются планировщиком *APScheduler* внутри серверного сервиса. Отдельный брокер сообщений, как у *Celery*, для пяти задач на одном сервере не требуется, а системный *cron* не даёт запретить повторный запуск задачи, пока предыдущий ещё не завершился [12].

3.1.7 Вспомогательные библиотеки

Запросы к площадке выполняет асинхронный клиент *aiohttp*, бот обращается к серверному интерфейсу через *httpx*. Нечёткое сравнение строк при проверке названия товара выполняет *rapidfuzz*. Отчёты в формате *Excel* формирует *openpyxl*. Журнал работы с ротацией файлов ведёт *loguru*. Тесты написаны на *pytest*.

3.1.8 Развёртывание

Все компоненты запускаются в контейнерах *Docker* и описаны одним файлом *Docker Compose*: СУБД, серверный сервис, бот, резервное копирование, служба мониторинга контейнеров и почтовый сервер для проверки писем. Такой способ воспроизводит окружение на любом сервере одной командой, автоматически перезапускает упавшие контейнеры и хранит данные базы в отдельном томе, который не пропадает при пересборке [13].

3.2 Реализация базы данных

3.2.1 Физическая схема

Физическая схема получена из логической (рисунок 2.2) по обычным правилам: каждая сущность стала таблицей, каждый атрибут – столбцом, связи «один ко многим» выражены внешними ключами в таблице на стороне

«многие». Физическая схема базы данных приведена на рисунке 3.1, назначение таблиц и характер их роста – в таблице 3.2.

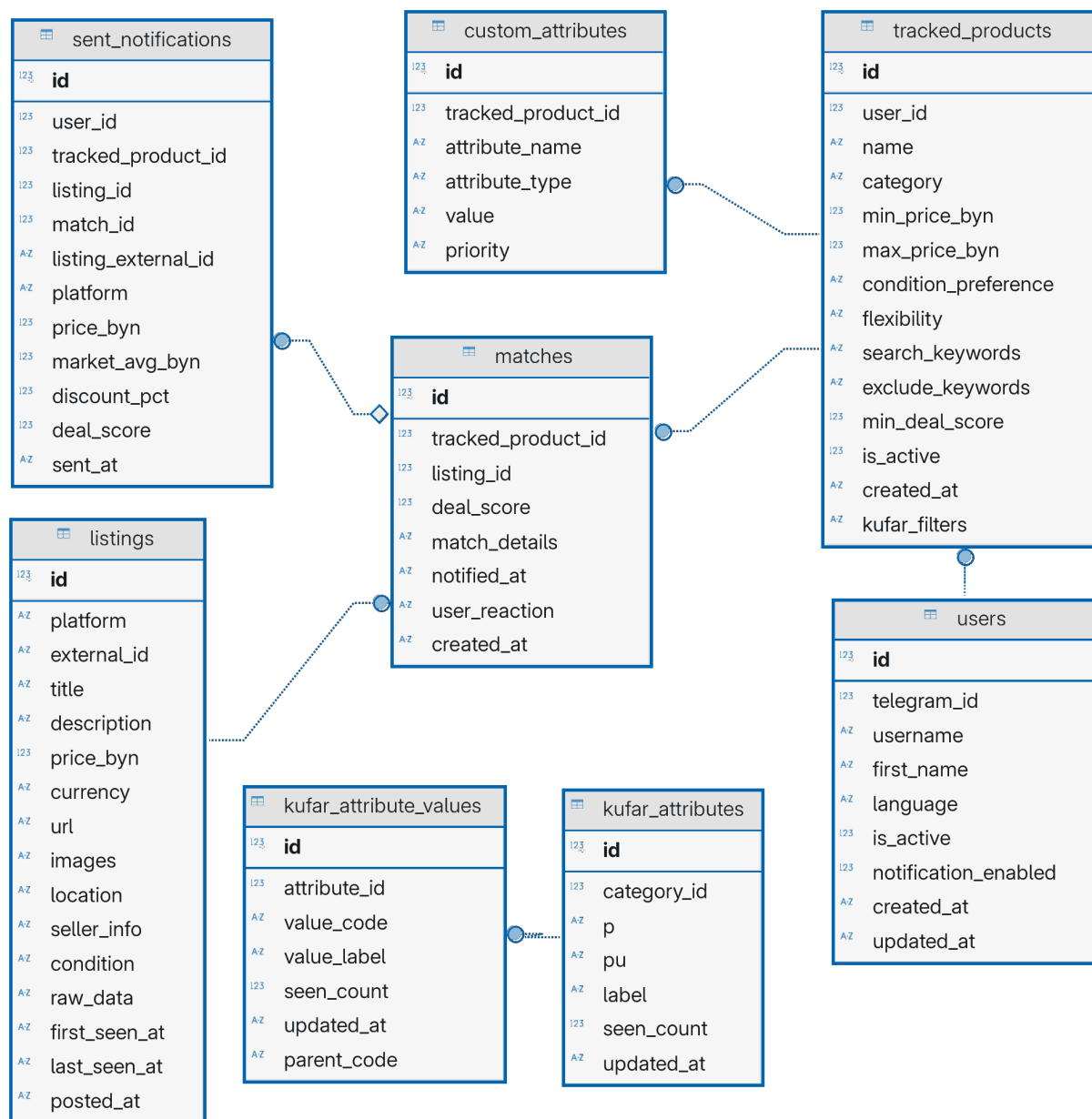


Рисунок 3.1 – Физическая схема базы данных

Типы данных выбраны по смыслу атрибутов. Первичными ключами всех таблиц служат целые числа с автоматическим приращением. Идентификатор пользователя *Telegram* хранится 64-битным целым: значения таких идентификаторов уже выходят за пределы 32-битного типа. Строки с известной верхней границей длины, например имя, код характеристики или адрес электронной почты, хранятся в строковых полях ограниченной длины, до 255 символов. Заголовок, описание и ссылка объявления не имеют разумного предела и хра-

Таблица 3.2 – Таблицы базы данных

Таблица	Назначение	Рост
Пользователи	Учётные записи, настройки, электронная почта	По одной строке на пользователя
Отслеживаемые товары	Критерии поиска пользователей	Не более 25 строк
Объявления	Сведения, полученные с площадки	С каждым циклом опроса, очищается через 30 дней
Совпадения	Подошедшие объявления и очередь доставки	По мере отбора
Журнал уведомлений	Факты отправки с ценой	С каждой отправкой, очищается через 90 дней
Атрибуты фильтров	Характеристики категорий площадки	Пересобирается раз в неделю
Значения атрибутов	Возможные значения характеристик	Пересобирается раз в неделю
Пользовательские атрибуты	Резерв для расширения системы	Не заполняется

няться в текстовых полях без ограничения длины. Хеш кода подтверждения занимает ровно 64 символа, по длине шестнадцатеричной записи результата *SHA-256*.

Цены хранятся числами с плавающей точкой двойной точности. В системе цены только сравниваются с порогом и между собой, а доля выгоды округляется до четырёх знаков, поэтому погрешность представления на результат не влияет. Если в систему будут добавлены суммирование или денежные расчёты, цены следует перевести в десятичный тип с фиксированной точностью.

Моменты времени хранятся без часового пояса и всегда в *UTC*. Время публикации, которое площадка передаёт с указанием пояса, система приводит к *UTC* перед записью. Благодаря этому сравнение «объявление опубликовано после добавления товара» корректно при любой настройке часового пояса сервера. Для дат создания и изменения записей значение по умолчанию задаёт сама СУБД текущим временем.

Поля с фиксированным набором значений (категория товара, площадка, реакция пользователя) объявлены перечислимыми типами *PostgreSQL*. СУБД

отклоняет любое значение вне перечня, и опечатка в коде не может записать в базу несуществующую категорию. Признаки хранятся логическим типом, а счётчики попыток целыми числами.

Тип *JSON* использован в семи столбцах: обязательные слова, слова-исключения и фильтры площадки у товара, фотографии, продавец и исходный ответ у объявления, детали события у совпадения. У этих данных общее свойство: их всегда читают и записывают целиком и никогда не ищут в базе по отдельному элементу. Фильтры площадки, например, подставляются в запрос к *Kufar.by* в неизменном виде.

3.2.2 Нормализация

Схема приведена к третьей нормальной форме с несколькими осознанными отступлениями.

Первая нормальная форма требует атомарности значений. Формально её нарушают *JSON*-столбцы: список слов-исключений или набор фильтров хранится в одной ячейке. Вынесение их в отдельные таблицы дало бы одну-две лишние таблицы на каждый список и соединение при каждом чтении товара, но ни одного нового запроса, которому такая структура была бы нужна. Система не ищет товары по слову-исключению и не считает, сколько пользователей выбрали бренд. Когда такие запросы появятся, фильтры можно будет перенести в отдельную таблицу связи между товаром и значением атрибута без изменения остальной схемы.

Вторая нормальная форма выполняется автоматически: все таблицы имеют простой суррогатный первичный ключ, и частичной зависимости от части ключа возникнуть не может.

Третья нормальная форма требует, чтобы неключевые атрибуты не зависели друг от друга. Это требование намеренно нарушено в журнале уведомлений. Площадка, внешний идентификатор объявления и цена в нём дублируют данные таблицы объявлений. Дублирование сделано по двум причинам. Проверка повтора (отправлялось ли это объявление пользователю и по какой цене) выполняется для каждого уведомления, и без копии каждая такая проверка требовала бы соединения с таблицей объявлений. Кроме того, в журнале записана цена на момент отправки, а в таблице объявлений хранится текущая цена, которая со временем меняется. Это разные факты, поэтому формально зависимость отсутствует, хотя названия столбцов совпадают.

Данные подтверждения электронной почты (хеш кода, срок, число попыток) хранятся в таблице пользователей, хотя по смыслу образуют отдельный объект. Пользователь связан с ними отношением один к одному, данные не-

лики и всегда читаются вместе с пользователем, поэтому отдельная таблица только добавила бы соединение.

3.2.3 Индексы

Каждый индекс ускоряет чтение и замедляет запись: при вставке и обновлении строки СУБД перестраивает все индексы таблицы. Таблица объявлений обновляется на каждом цикле опроса, поэтому индексы подбирались не по столбцам, а по запросам, которые система действительно выполняет. Все индексы – сбалансированные деревья (*B-tree*), принятые в *PostgreSQL* по умолчанию: они обслуживают и поиск по равенству, и сравнение по диапазону, которое нужно при подсчёте уведомлений за период. Соответствие индексов запросам приведено в таблице 3.3.

Таблица 3.3 – Индексы и обслуживаемые ими запросы

Индекс	Запрос	Частота
Объявления: площадка и внешний идентификатор (составной)	Поиск сохранённого объявления при обработке ответа площадки	До 1 050 раз за цикл
Пользователи: идентификатор <i>Telegram</i> (уникальный)	Определение пользователя по входящему сообщению	Каждое сообщение
Товары: владелец	Список товаров пользователя, подсчёт занятости личного лимита	Каждый диалог и каждое добавление
Совпадения: товар	Выборка очереди доставки, отчёт за период	Каждую минуту
Совпадения: объявление	Проверка, есть ли у объявления совпадения, при очистке	Раз в сутки
Журнал: получатель	Подсчёт уведомлений пользователя за сутки	Каждая отправка
Журнал: товар	Подсчёт уведомлений по товару за час	Каждая отправка
Журнал: внешний идентификатор объявления	Проверка, отправлялось ли объявление и по какой цене	Каждая отправка
Справочник: категория площадки, атрибут	Построение меню фильтров, обновление справочника	Каждое открытие фильтров

Самый частый запрос – поиск объявления по паре «площадка – внешний

идентификатор». Поиск выполняется для каждого объявления в каждом цикле: при 25 товарах и 42 объявлениях на запрос это до 1 050 поисков за цикл и более 500 000 в сутки. Для него создан составной индекс по двум столбцам. До его появления на этих столбцах были только отдельные индексы, и планировщику запросов приходилось объединять их результаты. Порядок столбцов в составном индексе выбран так, чтобы индекс годился и для запросов по одной площадке: ведущим стоит столбец площадки, а внешний идентификатор уточняет запись внутри неё. Поскольку площадка в системе пока одна, отдельный индекс по ней стал избыточным: выборка не сужается, а запись замедляется, поэтому такой индекс можно удалить.

Уникальный индекс по идентификатору *Telegram* решает две задачи сразу. Индекс поддерживает ограничение уникальности и обслуживает запрос, который выполняется чаще всех остальных запросов бота: на каждое входящее сообщение система находит пользователя по идентификатору его учётной записи. Без индекса этот поиск был бы последовательным просмотром всей таблицы пользователей.

Индекс по владельцу в таблице товаров обслуживает два запроса: вывод списка товаров пользователя и подсчёт занятости личного лимита перед добавлением нового товара. Оба выполняются с условием равенства по владельцу, и выборка по индексу возвращает сразу нужные несколько строк.

В таблице совпадений проиндексированы оба внешних ключа, но обслуживают они разные запросы. Индекс по товару соединяет совпадения с товарами при выборке очереди доставки и при формировании отчёта за период. Индекс по объявлению нужен задаче очистки: перед удалением устаревшего объявления система проверяет, не ссылается ли на него хотя бы одно совпадение, и без индекса такая проверка означала бы просмотр всей таблицы совпадений для каждого удаляемого объявления.

Три индекса журнала уведомлений поддерживают антиспам. По получателю считаются уведомления пользователя за текущие сутки, по товару – уведомления за последний час, по внешнему идентификатору объявления проверяется, отправлялось ли объявление этому пользователю и по какой цене. Все три запроса выполняются перед каждой отправкой, поэтому их стоимость напрямую влияет на длительность цикла доставки. Запрос проверки повтора отбирает строки по получателю, площадке и внешнему идентификатору одновременно; сейчас планировщик использует один из индексов и проверяет остальные условия по найденным строкам, а при росте журнала выгоднее станет составной индекс по получателю и внешнему идентификатору.

Справочник фильтров индексируется по категории площадки в табли-

це атрибутов и по атрибуту в таблице значений. Эти индексы обслуживают построение меню фильтров в боте, а уникальные ограничения на пару «категория – системное имя» и пару «атрибут – код значения» дополнительно ускоряют еженедельное обновление справочника: при повторном сборе система ищет существующую запись именно по этим парам.

Индексов по цене и категории в таблицах объявлений и товаров нет, и это сделано сознательно. Отбор объявлений по цене и категории выполняет сама площадка: система передаёт порог и категорию в поисковом запросе и получает только подходящие объявления. В базе данных по цене объявления не ищут. Таблица товаров содержит не более 25 строк, и последовательный просмотр такой таблицы быстрее обращения к индексу.

При росте нагрузки естественным улучшением станет частичный индекс по совпадениям, ещё не доставленным пользователю. Очередь доставки выбирает только их, а таких записей в любой момент единицы, тогда как в таблице совпадений хранится вся история. Частичный индекс занимает место, пропорциональное числу недоставленных совпадений, и не растёт вместе с историей.

3.2.4 Ограничения целостности

Целостность данных обеспечивается ограничениями на уровне СУБД и проверками на входе в серверный интерфейс.

На уровне СУБД действуют:

- первичные ключи во всех таблицах;
- внешние ключи с явно заданными правилами удаления: товар ссылается на пользователя, совпадение на товар и объявление, запись журнала на пользователя, товар, объявление и совпадение, значение атрибута на атрибут;
- ограничения уникальности: идентификатор *Telegram* пользователя, пара «категория – системное имя» у атрибута фильтра, пара «атрибут – код значения» у значения;
- ограничения *NOT NULL* на всех обязательных столбцах; необязательными оставлены только данные, которых действительно может не быть: цена договорного объявления, адрес почты, время доставки недоставленного совпадения;
- перечислимые типы для категорий, площадок и реакций;
- значения по умолчанию на стороне СУБД для дат создания и изменения записей и для столбцов, добавленных миграциями; начальные значения признаков задаёт приложение при вставке записи.

Ограничения уникальности справочника позволяют пересобирать его

без дублей: при повторном сборе существующее значение обновляется, а не добавляется второй раз. Совпадение для пары «товар – объявление» создаётся только после проверки, что такого совпадения ещё нет. Проверка и вставка выполняются в одной транзакции цикла опроса.

Удаление товара затрагивает зависимые записи, поэтому правило удаления задано в схеме для каждого внешнего ключа. Совпадения, пользовательские атрибуты и записи журнала уведомлений по товару удаляются вместе с ним (*ON DELETE CASCADE*). Ссылка журнала на совпадение обнуляется (*ON DELETE SET NULL*): запись журнала хранит собственную копию цены и внешнего идентификатора объявления и без совпадения не теряет смысла. По тому же правилу вместе с пользователем удаляются его товары и журнал, а вместе с атрибутом фильтра – его значения. У ссылки журнала на объявление правила удаления нет: очистка удаляет только объявления без совпадений, а на них журнал не ссылается. Правила выполняет сама СУБД, поэтому одного запроса *DELETE* к таблице товаров достаточно, чтобы удалить товар со всеми зависимыми записями в одной транзакции.

Проверки, которые неудобно выразить ограничениями СУБД, выполняются при приёме данных серверным интерфейсом: пороговая цена больше нуля и не больше 10 000 000 рублей, название не длиннее 255 символов, не более 20 слов в каждом списке и не более 100 символов в слове, коды фильтров принадлежат категории товара. Файл *CSV* при импорте ограничен 256 КБ и 100 строками. Некорректный запрос отклоняется с понятным сообщением до обращения к базе данных.

Каждый запрос к серверному интерфейсу выполняется в одной транзакции: при успехе изменения фиксируются, при любой ошибке транзакция откатывается целиком. Цикл опроса площадки записывает объявления и совпадения всех товаров одной транзакцией. Доставка уведомлений отмечает совпадение доставленным и добавляет запись в журнал в одной транзакции, поэтому отметка без записи или запись без отметки появиться не могут.

3.2.5 Миграции и обслуживание данных

Схема базы данных изменялась пятью миграциями. Первая создаёт все таблицы и индексы. Вторая добавляет в таблицу совпадений счётчик попыток доставки и текст ошибки, на которых построена повторная отправка. Третья добавляет составной индекс по площадке и внешнему идентификатору объявления. Четвёртая добавляет поля электронной почты в таблицу пользователей и отдельную почтовую очередь в таблицу совпадений. Пятая задаёт правила удаления для внешних ключей и удаляет избыточный индекс по площадке.

Новые обязательные столбцы добавлялись со значением по умолчанию на стороне СУБД, поэтому миграции применялись к таблицам, уже содержащим данные, без их пересоздания.

Миграции применяет серверный сервис при запуске. Перед применением сервис берёт рекомендательную блокировку *PostgreSQL*: если одновременно запускаются два экземпляра сервиса, второй дожидается, пока первый закончит миграцию, и находит схему уже в актуальном состоянии.

Объявления сохраняются на каждом цикле опроса, и без очистки их таблица росла бы всё время работы системы. Раз в сутки удаляются объявления, которые не появлялись в выдаче 30 дней и не участвуют ни в одном совпадении. Объявления с совпадениями сохраняются, потому что на них ссылается история уведомлений и отчёты пользователя. Записи журнала уведомлений старше 90 дней удаляются по тому же расписанию.

3.2.6 Резервное копирование

Резервное копирование выполняет отдельный контейнер. Раз в сутки контейнер создаёт полный логический дамп базы данных утилитой *pg_dump*, сжимает его и сохраняет в каталог на сервере вне тома СУБД. Хранятся семь последних копий, более старые удаляются автоматически.

Резервная копия, которую ни разу не восстанавливали, не даёт гарантии, что данные удастся вернуть. Поэтому в систему входит сценарий проверки восстановления. Сценарий создаёт временную базу данных, загружает в неё последний дамп, подсчитывает строки в основных таблицах и удаляет временную базу. Проверка запускается вручную одной командой и не затрагивает рабочую базу.

Данные СУБД хранятся в именованном томе *Docker*. Пересборка и перезапуск контейнеров их не затрагивают, а резервные копии в отдельном каталоге защищают от повреждения самого тома.

3.3 Применение интеллектуальных инструментов в инженерном проектировании

Под интеллектуальными инструментами здесь понимаются большие языковые модели, которые принимают постановку задачи на естественном языке и возвращают текст, код или описание схемы. При разработке системы применялась модель *Claude* в среде *Claude Code*, работающая с файлами проекта. Инструмент использовался как помощник на трёх этапах: при проектировании модели данных, при написании кода и при подготовке графического мате-

риала. Решение о применении каждого предложенного варианта принимал разработчик после проверки.

3.3.1 Порядок работы с инструментом

Работа строилась по одной схеме. Задача формулировалась вместе с контекстом: фрагментом схемы базы данных, текстом запроса или сообщением об ошибке. Полученный ответ не принимался как готовый результат, а проверялся: код – запуском тестов, запросы – планом выполнения, схема данных – сверкой с требованиями подраздела 1.3. Ответы, которые не удавалось воспроизвести или обосновать, отбрасывались.

Такой порядок выбран из-за особенности инструмента: модель порождает синтаксически верный и правдоподобный результат, но не проверяет его на данных системы. Ошибка в ответе выглядит так же убедительно, как верное решение, поэтому проверка обязательна для каждого предложения.

3.3.2 Решённые задачи

Задачи, при решении которых применялся инструмент, сведены в таблицу 3.4.

3.3.3 Ошибки, найденные при проверке

Проверка результатов выявила два дефекта, которые относятся непосредственно к базе данных и были исправлены отдельными миграциями.

1. Правила удаления связанных записей существовали только в коде приложения, а в схеме базы данных отсутствовали. Удаление товара, по которому уже отправлено уведомление, завершалось нарушением ссылочной целостности. Миграция добавила правила в саму схему: для товаров, совпадений и значений справочника задано каскадное удаление, а ссылка журнала на совпадение обнуляется, поскольку запись журнала хранит собственную копию цены и идентификатора объявления и должна пережить удаление совпадения.

2. Среди индексов таблицы объявлений оказался избыточный: отдельный индекс по площадке дублировал составной индекс по паре «площадка – внешний идентификатор», который начинается с того же столбца и обслуживает те же запросы. Избыточный индекс удалён, потому что он замедлял вставку объявлений и занимал место, не ускоряя ни один запрос.

Оба дефекта не были обнаружены ни тестами, ни разбором кода: первый проявляется только при удалении товара с историей уведомлений, второй вообще не влияет на правильность результата. Нашла их сверка схемы с перечнем запросов цикла, выполненная вручную.

Таблица 3.4 – Задачи, решённые с применением интеллектуальных инструментов

Задача	Результат работы инструмента	Что потребовало проверки
Проектирование модели данных	Черновой состав сущностей и связей по словесному описанию предметной области	Мощность связей, состав ключей, выделение совпадения в отдельную сущность
Подбор индексов	Перечень индексов под запросы цикла опроса	Соответствие плану выполнения запроса, поиск избыточных индексов
Разбор ответа площадки	Черновой код разбора полей объявления и характеристик категории	Поведение на объявлениях без цены и без описания
Миграции схемы	Заготовки миграций с прямым и обратным переходом	Правила удаления связанных записей, порядок применения
Тесты	Наборы проверок для отбора объявлений и лимитов уведомлений	Полнота граничных случаев
Графический материал	Исходные описания схем и диаграмм для <i>Graphviz</i> и <i>PlantUML</i>	Соответствие ГОСТ 19.701–90 и фактическому коду

3.3.4 Оценка применимости

Инструмент даёт выигрыш на задачах, где ответ проверяется быстро и однозначно: черновик кода разбора ответа площадки, заготовка миграции, набор тестов, описание диаграммы для *Graphviz*. Время на такие задачи сокращается заметно, потому что проверка занимает меньше времени, чем написание с нуля.

На задачах, где решение зависит от особенностей площадки и характера нагрузки, выигрыш оказался небольшим. Ограничение частоты запросов, стратегия пауз после отказа площадки, порог выгодности и правила повторной отправки уведомления подбирались по наблюдениям за работой системы, а не по предложенному варианту. Модель не располагает сведениями о поведении конкретной площадки и о распределении цен в категориях, поэтому её предложения в этой части оставались общими.

Отсюда следует правило, которого придерживались при разработке: интеллектуальный инструмент применим там, где результат проверяем формально, и не применим там, где решение опирается на данные, которых у инструмента нет. Ответственность за принятое решение остаётся на разработчике в обоих случаях.

3.4 Программная реализация системы

3.4.1 Архитектура системы

Система состоит из нескольких сервисов, каждый из которых запущен в отдельном контейнере на одном сервере. Архитектура развёртывания приведена на рисунке 3.2.

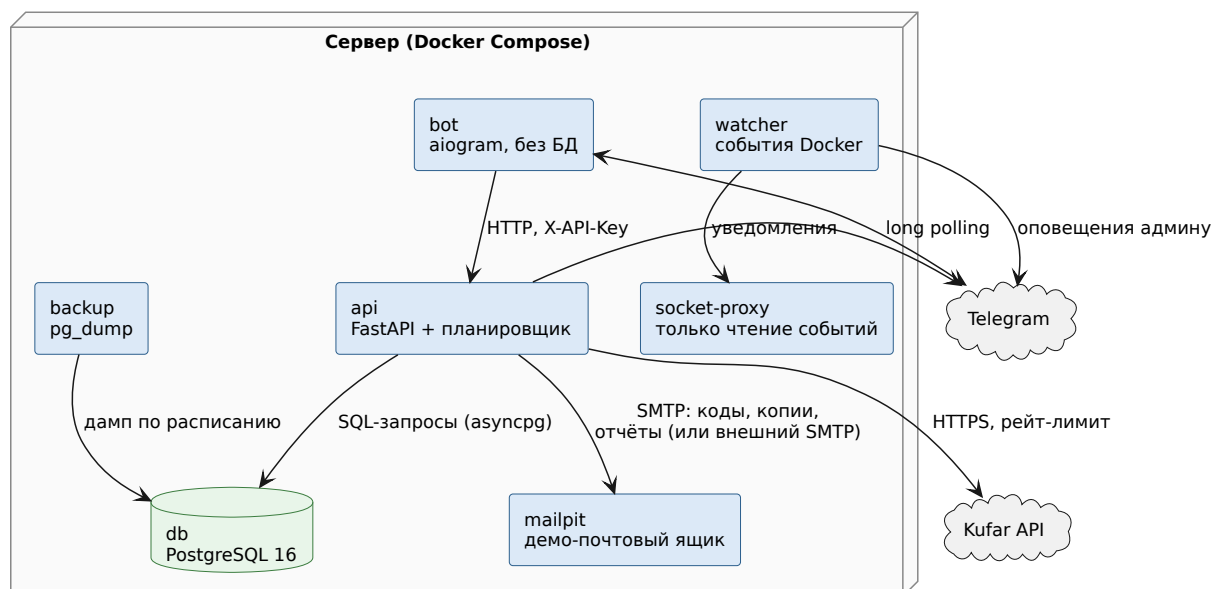


Рисунок 3.2 – Архитектура развёртывания системы

Основную работу выполняют три сервиса. СУБД *PostgreSQL* хранит все данные. Серверный сервис владеет базой данных: только этот сервис выполняет SQL-запросы, применяет миграции и запускает периодические задачи. Бот отвечает только за диалог с пользователем в *Telegram*. Собственного доступа к базе данных у бота нет: все данные для показа и изменения бот получает от серверного сервиса по *HTTP*, прикладывая к каждому запросу общий секретный ключ.

Такое разделение выбрано ради данных. Когда в базу пишет только один сервис, все правила записи собраны в одном месте. Лимит отслеживаемых товаров, например, проверяется в серверном сервисе, и обойти его нельзя ни

через бот, ни через импорт файла, ни прямым запросом к интерфейсу. Схемой базы данных управляет тот же сервис, поэтому несогласованных изменений структуры не бывает.

Уведомления серверный сервис отправляет в *Telegram* сам, не через бота. Для отправки сообщений достаточно токена бота, а получать входящие сообщения продолжает сервис бота. Поэтому остановка бота, например при обновлении, не прерывает доставку уведомлений.

Вспомогательные сервисы обеспечивают надёжность. Сервис резервного копирования раз в сутки выгружает дампы базы. Служба мониторинга следит за событиями контейнеров и сообщает администратору о запуске, падении и потере работоспособности любого из них. События *Docker* служба читает через отдельный посредник, который открывает к ним доступ только на чтение: прямой доступ к управляющему сокету *Docker* равносителен правам администратора сервера. Почтовый сервер-ловушка принимает все письма системы и показывает их в веб-интерфейсе, что упрощает проверку почтовых функций без настоящего почтового ящика.

Внутри серверного сервиса код разделён на слои. Модули обработки и их связи с сущностями данных показаны на рисунке 3.3.

Сущности данных описывают содержимое таблиц базы данных:

- *User* – пользователь: номер учётной записи *Telegram*, язык интерфейса, признаки активности и уведомлений, адрес электронной почты и признак его подтверждения;
- *TrackedProduct* – отслеживаемый товар: название, категория, границы цены, списки слов поиска и исключений, фильтры площадки в поле *kufar_filters* и порог выгодности *min_deal_score*;
- *Listing* – объявление, полученное с площадки: площадка, внешний идентификатор, заголовок, описание, цена, ссылка, время публикации, время первого и последнего появления в выдаче;
- *Match* – совпадение объявления с отслеживаемым товаром: показатель выгоды, подробности отбора, время доставки *notified_at*, число попыток *notify_attempts* и текст ошибки;
- *SentNotification* – запись журнала отправленных уведомлений с ценой объявления на момент отправки;
- *KufarAttribute* и *KufarAttributeValue* – справочник характеристик категорий площадки и их допустимых значений.

Доступ к площадке построен на наследовании. Абстрактный класс *BaseScraper* хранит сетевую сессию, удерживает частоту запросов методом *_rate_limit* и выполняет запрос методом *_get*, а поиск и загрузку отдельно-

го объявления объявляет, но не реализует. Класс *KufarScraper* реализует их для *Kufar.by*: метод *_build_params* собирает параметры запроса по стратегии категории, методы *_parse_search_response* и *_parse_ad* превращают ответ площадки в структуры *ScrapedListing*. Добавление второй площадки сводится к новому наследнику *BaseScraper*, потому что остальной код работает со структурой *ScrapedListing* и об устройстве площадки ничего не знает.

Обработка данных вынесена в отдельные модули и классы, показанные на рисунке 3.3.

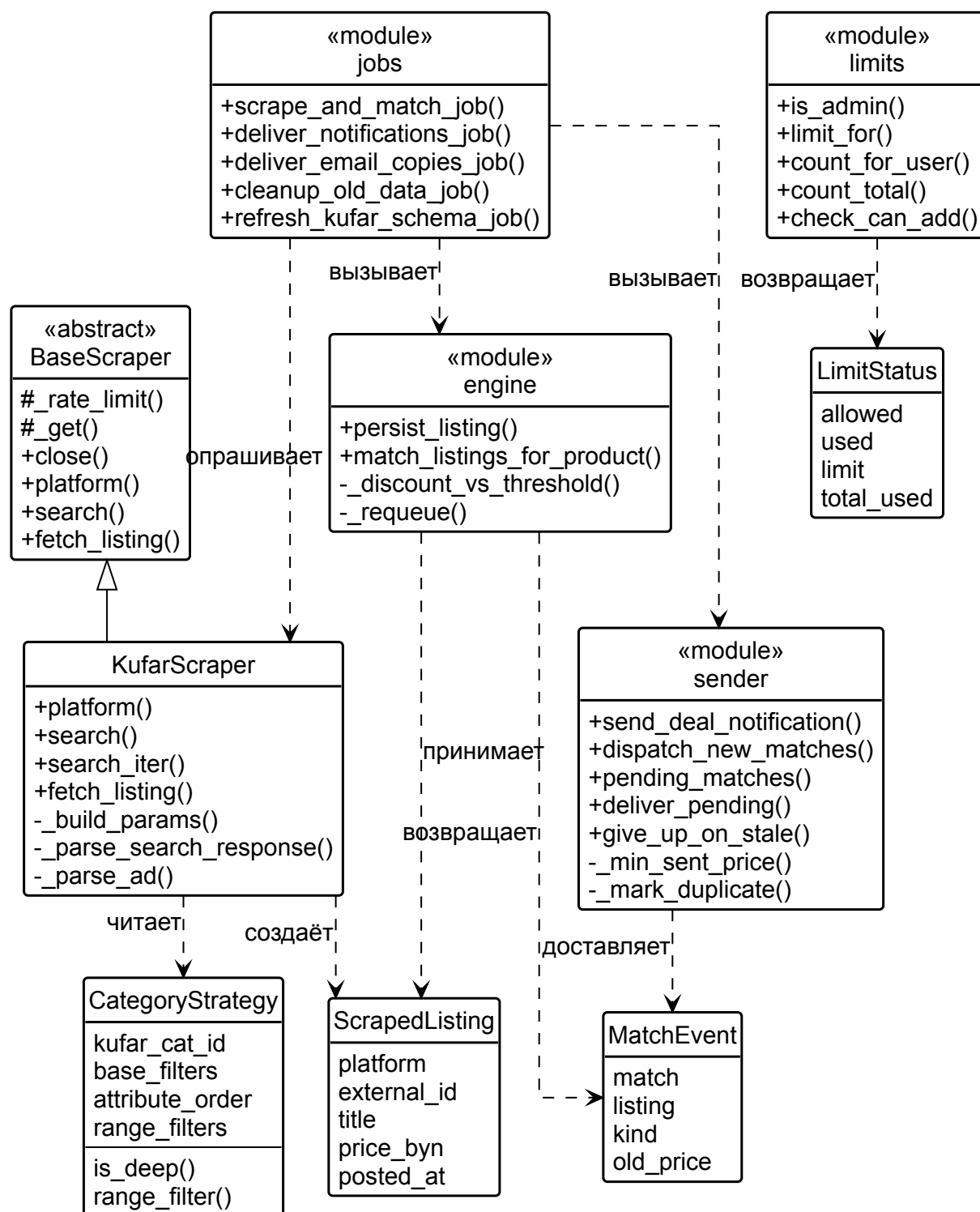


Рисунок 3.3 – Диаграмма классов модулей обработки

Назначение модулей:

– *engine* – функция *persist_listing* добавляет или обновляет объявление и возвращает прежнюю цену, функция *match_listings_for_product* отбирает объявления по критериям товара, создаёт совпадения и возвращает список событий *MatchEvent*;

– *sender* – функция *deliver_pending* разбирает очередь недоставленных совпадений, функция *send_deal_notification* отправляет одно уведомление, функция *_min_sent_price* проверяет журнал на повтор, функция *give_up_on_stale* закрывает совпадения старше трёх суток;

– *limits* – функция *check_can_add* проверяет лимит отслеживаемых товаров и возвращает структуру *LimitStatus*, функции *is_admin* и *limit_for* определяют потолок для конкретного пользователя;

– *jobs* – периодические задачи планировщика: опрос площадки *scrape_and_match_job*, доставка уведомлений *deliver_notifications_job*, почтовые копии *deliver_email_copies_job*, очистка устаревших данных *cleanup_old_data_job* и обновление справочника *refresh_kufar_schema_job*.

В структуре *CategoryStrategy* собраны правила отбора для каждой категории: код категории площадки, обязательные фильтры, порядок характеристик и их числовые диапазоны. Результаты работы модули передают друг другу структурами: *ScrapedListing* – разобранное объявление площадки, *MatchEvent* – найденное совпадение вместе с прежней ценой и типом события, *LimitStatus* – результат проверки лимита. На диаграммах не показаны части системы, которые не работают с данными напрямую: обработчики бота, маршруты программного интерфейса, словари локализации и формирование книг *Excel*.

Слои обращаются друг к другу сверху вниз. Маршрут программного интерфейса проверяет входные данные и вызывает сервис бизнес-логики. Сервис выполняет SQL-запросы к базе данных и возвращает результат. Модуль *jobs* обращается к тем же модулям *engine*, *sender* и *limits*, что и маршруты, поэтому одно и то же правило, например проверка повтора уведомления, реализовано один раз. Класс *KufarScraper* о базе данных ничего не знает и возвращает объявления структурами *ScrapedListing*, а сохраняет их функция *persist_listing*.

3.4.2 Цикл опроса площадки и обработка объявлений

Цикл опроса запускается планировщиком каждые 3 минуты. Одновременно может выполняться только один цикл: если предыдущий ещё не завершился, следующий запуск пропускается. Цикл выполняет следующие шаги:

- 1 Проверить паузу. Если после отказа площадки ещё остались пропускаемые циклы, уменьшить их счётчик и завершить цикл.

- 2 Выбрать из базы данных все активные отслеживаемые товары.

- 3 Для каждого товара по очереди:

- пропустить товар, если у него не задана пороговая цена;

- по стратегии категории сформировать запрос: код категории площадки, тип сделки, обязательные фильтры, фильтры пользователя, название и ограничение цены сверху, равное порогу;
- дожидаться разрешения ограничителя частоты и отправить запрос;
- если площадка ответила кодом 403 или 429, прервать цикл целиком;
- передать полученные объявления модулю отбора.

4 Зафиксировать транзакцию.

5 Если цикл занял больше 80 % интервала, записать предупреждение в журнал работы.

Ограничитель частоты работает в два слоя. Перед каждым запросом выдерживается пауза около 2 секунд со случайной добавкой от 0,5 до 1,5 секунды, чтобы запросы не шли с одинаковым интервалом. Кроме того, ограничитель помнит время запросов за последнюю минуту и, если их уже 30, ждёт, пока самый старый выйдет из этого окна. При отказе площадки пауза до следующего опроса удваивается с каждым отказом подряд: 1, 2, 4, 8, 16 циклов. Администратор получает одно оповещение, а повторные оповещения того же вида в течение 30 минут подавляются.

Модуль отбора обрабатывает каждое полученное объявление так:

1 Найти объявление в базе данных по паре «площадка – внешний идентификатор».

2 Если объявление найдено, запомнить его прежнюю цену и обновить запись новыми данными, иначе добавить новую запись.

3 Проверить релевантность. Объявление отклоняется, если выполняется хотя бы одно условие:

- цена отсутствует, не ниже порога или ниже минимальной правдоподобной цены категории;
- заголовок содержит стоп-слово категории (для электроники это «чехол», «защитное стекло» и подобные);
- заголовок или описание содержат слово-исключение пользователя;
- в заголовке или описании нет какого-либо слова из названия товара (в категориях с фильтрами площадки проверка названия не выполняется);
- нет какого-либо обязательного слова.

4 Если совпадение для этой пары «товар – объявление» уже есть, обновить его только при снижении цены.

5 Иначе определить тип события: «новое», если объявление опубликовано после добавления товара, «снижение цены», если цена упала с прошлого опроса. Если ни то ни другое, объявление пропускается.

6 Создать совпадение с показателем выгоды и пустым временем доставки.

Проверка названия допускает расхождения в написании. Слово из названия ищется в три этапа: сначала как подстрока текста объявления, затем по словарю известных вариантов записи (латиница и кириллица, распространённые сокращения), затем нечётким сравнением с каждым словом текста. На последнем этапе слово длиной от трёх символов считается найденным, если мера сходства на основе редакционного расстояния составляет не менее 85 %.

Отдельно работает мгновенный поиск. Сразу после добавления товара система выполняет тот же запрос к площадке и те же проверки релевантности, сортирует прошедшие объявления по цене и показывает пользователю пять самых дешёвых. В базу данных эти объявления не записываются: мгновенный поиск показывает состояние рынка, а отслеживание начинается с момента добавления товара.

3.4.3 Механизм доставки уведомлений

Уведомления доставляются по схеме исходящей очереди (*outbox*). Цикл опроса только записывает совпадения в базу данных, а отправляет их отдельная задача, которую планировщик запускает каждую минуту. Очередью служит сама таблица совпадений: в очереди находятся записи с пустым временем доставки. Отдельной таблицы или брокера сообщений для этого не нужно: запись совпадения и постановка в очередь выполняются одной операцией в одной транзакции.

Задача доставки выполняет следующие шаги:

1 Выбрать до 50 совпадений в порядке создания, для которых одновременно:

- время доставки пусто;
- неудачных попыток меньше пяти;
- совпадение создано не раньше трёх суток назад;
- товар активен, пользователь активен и не отключил уведомления.

2 Сгруппировать совпадения по товарам.

3 Для каждого товара посчитать, сколько уведомлений ещё можно отправить: не больше пяти за последний час по этому товару и не больше 30 за текущие сутки пользователю. Если лимит исчерпан, оставить совпадения в

очереди.

4 Отсортировать совпадения товара по цене, чтобы при ограниченном лимите первыми ушли самые выгодные.

5 Для каждого совпадения найти в журнале минимальную цену, по которой это объявление уже отправлялось пользователю:

- если такой отправки не было, отправить уведомление;
- если новая цена ниже минимальной, отправить уведомление с пометкой о снижении цены;
- иначе закрыть совпадение как дубликат.

6 После успешной отправки заполнить время доставки и добавить запись в журнал уведомлений.

7 Зафиксировать транзакцию.

Ошибки отправки обрабатываются по-разному. Если *Telegram* просит снизить частоту запросов, доставка прекращается до следующего запуска: это ограничение действует на бота целиком, и продолжать отправки другим пользователям бессмысленно. Пользователь, заблокировавший бота, помечается неактивным, и его совпадения перестают выбираться из очереди. При любой другой ошибке у совпадения увеличивается счётчик попыток и сохраняется текст ошибки, а само совпадение остаётся в очереди до следующего запуска. После пятой неудачной попытки совпадение из очереди исключается, а число таких совпадений записывается в журнал работы.

Почтовые копии ведут отдельную очередь в той же таблице совпадений, со своим временем отправки и счётчиком попыток. Копия отправляется только для совпадения, уже доставленного в *Telegram*, поэтому все проверки лимитов и повторов действуют и для почты. Раз в 5 минут задача собирает все новые доставленные совпадения пользователя, у которого подтверждён адрес и включены копии, и отправляет одно сводное письмо со списком до 20 объявлений. Сбой почтового сервера не задерживает доставку в *Telegram*, потому что две очереди обрабатываются разными задачами.

3.4.4 Лимиты отслеживания и права администратора

Каждый отслеживаемый товар добавляет один запрос к площадке за цикл опроса, поэтому число товаров ограничено двумя пределами:

- личный лимит: 3 товара для пользователя и 10 для администратора;
- общий лимит: не более 25 товаров на всю систему.

Личный лимит защищает от ситуации, когда один пользователь занимает всю систему. Общий лимит защищает площадку и саму систему: при интервале опроса 180 секунд и примерно 3 секундах на запрос с учётом паузы за цикл

можно обработать около 60 товаров. Предел в 25 товаров оставляет запас, чтобы цикл гарантированно завершился до начала следующего. При запуске серверный сервис сравнивает общий лимит с этой расчётной ёмкостью и предупреждает в журнале работы, если лимит задан больше, чем цикл успевает обработать.

Проверка лимитов выполняется дважды: перед началом диалога добавления товара и в момент сохранения. Вторая проверка нужна потому, что за время диалога другой пользователь мог занять последнее свободное место. Логика проверки такова:

- 1 Посчитать товары пользователя и товары всех пользователей.
- 2 Если товаров пользователя не меньше его личного лимита, отказать с пояснением «достигнут личный лимит».
- 3 Если товаров в системе не меньше 25, отказать с пояснением «достигнут общий лимит системы».
- 4 Иначе разрешить добавление.

Все пути добавления товара проходят через одну и ту же операцию сервиса товаров: пошаговый диалог, импорт из ссылки и импорт файла CSV. При импорте файла товары добавляются по строкам, пока лимит не исчерпан. В отчёте об импорте пользователь видит число добавленных товаров, ошибки по номерам строк и сообщение о том, какой лимит остановил импорт.

Права администратора определяются списком идентификаторов *Telegram* в конфигурации системы, отдельной таблицы ролей нет. Администраторов в системе единицы, и назначает их владелец сервера, изменяя конфигурацию. Администратор отличается от пользователя тремя возможностями: личным лимитом в 10 товаров, отчётом о состоянии системы и служебными оповещениями о блокировке со стороны площадки и о событиях контейнеров. Остальные функции у него те же, что у пользователя, и доступа к чужим товарам администратор не получает: любая операция с товаром выполняется только в пределах товаров владельца.

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для работы с системой не нужно устанавливать отдельное приложение: достаточно клиента *Telegram* на смартфоне, компьютере или в браузере. Пользователь находит бота по имени и отправляет команду */start*. Отдельной регистрации нет. При первом сообщении система сама создаёт учётную запись по идентификатору *Telegram*, а язык интерфейса выбирает по языку приложения *Telegram*: русский или английский.

В ответ на команду */start* бот присылает приветствие с кратким описанием работы и кнопки главного меню: «Мои товары», «Добавить товар», «Настройки» и «Помощь». Те же четыре кнопки закрепляются под полем ввода сообщения, так что главное меню доступно на любом экране. Главное меню показано на рисунке 4.1.



Рисунок 4.1 – Приветствие и главное меню бота

Кроме кнопок, бот понимает команды: */start* – главное меню, */add* – добавить товар, */my* – мои товары, */report* – отчёт в *Excel*, */settings* – настройки, */language* – язык интерфейса, */profile* – профиль, */help* – справка. Список команд продублирован в меню *Telegram* слева от поля ввода.

4.1 Роль администратора

Администратор – пользователь, идентификатор *Telegram* которого указан в конфигурации системы. Отдельного входа или пароля у администратора нет: бот распознаёт его по учётной записи *Telegram*. Чтобы назначить

администратора, владелец сервера добавляет его идентификатор в список администраторов в файле окружения и перезапускает сервисы. Узнать свой идентификатор можно в разделе профиля бота.

Администратор пользуется всеми функциями пользователя, описанными в подразделе 4.2. Возможности администратора отличаются в трёх местах.

4.1.1 Повышенный лимит отслеживания

Администратор может отслеживать до 10 товаров вместо 3. В профиле (команда */profile*) в строке «Статус» указано «администратор», а в строке «Лимит отслеживания» видно, сколько товаров занято у него самого и во всей системе. Общий предел в 25 товаров действует и для администратора и защищает площадку от перегрузки запросами. Профиль администратора показан на рисунке 4.2.

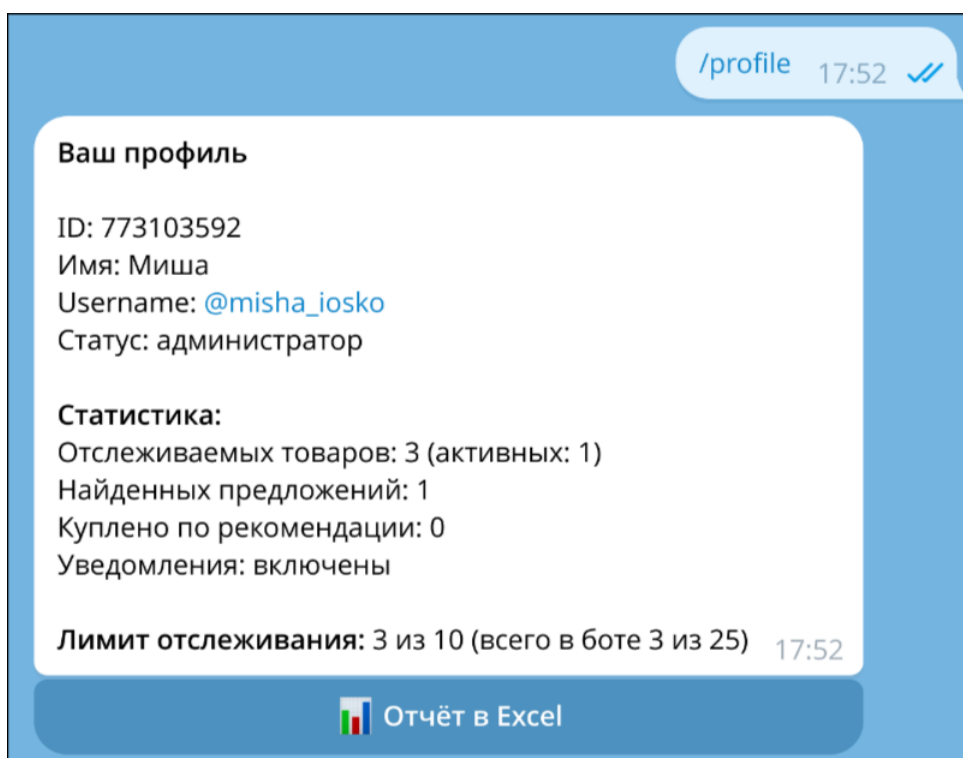


Рисунок 4.2 – Профиль администратора

4.1.2 Отчёт о состоянии системы

Под профилем расположена кнопка «Отчёт в *Excel*». В меню отчётов администратор, помимо кнопок периода, видит кнопку «Состояние системы (30 дн.)». По ней бот присылает книгу *Excel*, в которой собраны показатели за последние 30 дней:

- число пользователей всего и активных;

- число отслеживаемых товаров и из них активных;
- число совпадений, найденных за период, и доставленных уведомлений;
- число совпадений, ожидающих доставки, и не доставленных после всех попыток;
- распределение отслеживаемых товаров по категориям.

По этому отчёту администратор видит, справляется ли система с нагрузкой. Растущее число ожидающих доставки совпадений означает, что уведомления не успевают уходить, а ненулевое число недоставленных говорит о том, что часть пользователей не получает сообщения. Пример отчёта приведён на рисунке 4.3.

	А	В
1	Состояние системы	
2	Период	последние 30 дн.
3	Сформирован	21.09.2026 14:56
4		
5	Показатель	Значение
6	Пользователей всего	1
7	Активных пользователей	1
8	Отслеживаемых товаров	5
9	Из них активных	3
10	Совпадений за период	1
11	Доставлено за период	0
12	Ждут доставки	1
13	Не доставлено после всех попыток	0
14	Отправлено уведомлений за период	0
15		
16	Товары по категориям	
17	Категория	Товаров
18	laptop	2
19	apartment	2
20	sports	1

Рисунок 4.3 – Отчёт о состоянии системы

4.1.3 Служебные оповещения

Администратор получает от бота сообщения о событиях, которые требуют внимания человека:

- площадка ответила отказом (код 403 или 429) – в сообщении указано, на сколько циклов приостановлена проверка, и что делать, если отказ повторяется: снизить общий лимит товаров или увеличить интервал опроса;
- контейнер системы запущен, остановлен штатно или упал с ненулевым кодом выхода;
- контейнер перестал проходить проверку работоспособности.

Одно и то же оповещение не повторяется подряд. Сообщение о блокировке площадкой приходит не чаще раза в 30 минут, о событиях контейнера не чаще раза в 5 минут. Если контейнер перезапускается чаще, в оповещении

отдельно указано, что это похоже на цикл перезапусков. Пример оповещения показан на рисунке 4.4.

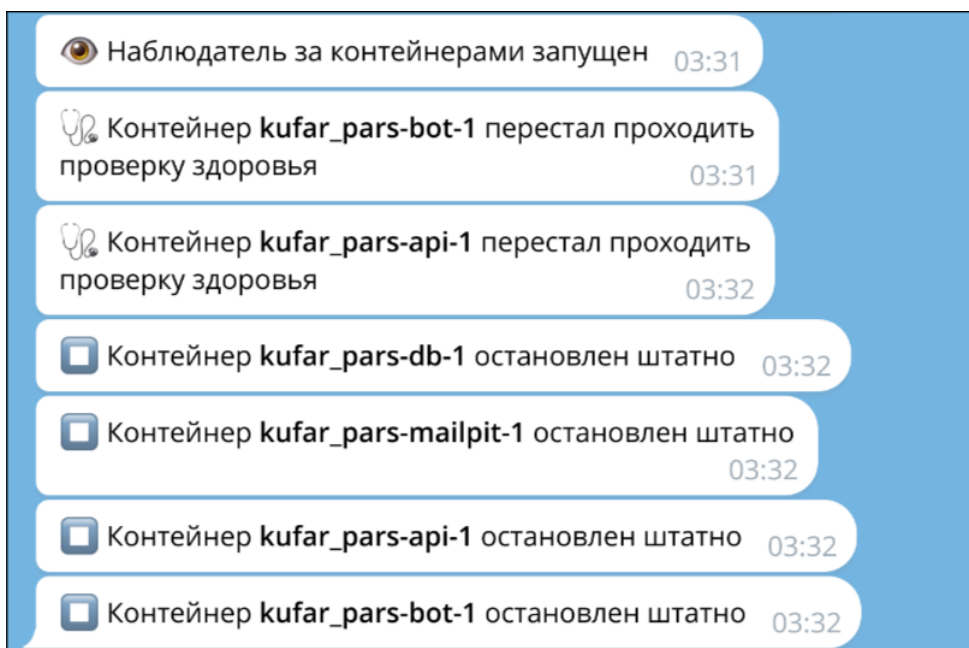


Рисунок 4.4 – Служебное оповещение администратору

4.1.4 Обслуживание сервера

Часть задач администратор выполняет не в *Telegram*, а на сервере. Все сервисы запускаются и обновляются одной командой *Docker Compose*, журналы работы доступны там же. Резервные копии базы данных создаются автоматически раз в сутки в отдельном каталоге сервера. Пригодность последней копии проверяется сценарием восстановления, который загружает её во временную базу и выводит число строк в основных таблицах. Письма системы при стандартной настройке попадают в почтовый сервер-ловушку, который доступен в браузере на сервере; для доставки настоящим адресатам в файле окружения указываются параметры внешнего почтового сервера.

4.2 Роль пользователя

4.2.1 Добавление отслеживаемого товара

Добавление начинается с кнопки «Добавить товар» или команды */add*. Бот предлагает выбрать категорию из 13 вариантов. Первыми в списке стоят «Велосипед», «Ноутбук» и «Квартира (аренда)»: в них отбор ведётся по фильтрам самого *Kufar.by* и получается заметно точнее. Над списком категорий находится кнопка «Вставить ссылку с *Kufar*»; работа с ней описана далее. Экран выбора категории показан на рисунке 4.5.



Рисунок 4.5 – Выбор категории товара

На следующем шаге вводится название товара: бренд и модель, например «*Trek Marlin*» или «*Pixel 7a*». Лишних слов писать не стоит: бот требует, чтобы в объявлении встретилось каждое слово названия. В категориях с фильтрами площадки название можно пропустить кнопкой «Пропустить», тогда отбор выполняют фильтры.

Для велосипедов, ноутбуков и квартир бот открывает меню фильтров. В нём перечислены характеристики категории: для велосипеда это производитель, класс, размер рамы, диаметр колёс, материал рамы, для кого, состояние, регион и город. Пользователь открывает характеристику и отмечает одно или несколько значений; объявление подойдёт, если в нём есть любое из отмеченных. Длинные списки, например бренды или процессоры, разбиты на страницы по восемь значений. Кнопка «Сбросить» снимает выбор, «Назад» возвращает к списку характеристик, «Готово» завершает настройку. Сначала лучше выбрать регион: после этого список городов и районов сократится до выбранного региона. Меню фильтров показано на рисунке 4.6.

Выберите нужные (в каждом можно отметить несколько значений) и нажмите «Готово».

17:57

☐ Комнат

☐ Микрорайон

☐ Ремонт

☐ Студия

☐ Балкон

☐ Санузел

☐ Материал стен

☐ Регион

☐ Город / Район

☐ Общая площадь, м²

☐ Этаж

☐ Год постройки

☒ Готово

Рисунок 4.6 – Выбор значений фильтра

У квартир есть числовые фильтры: общая площадь, этаж и год постройки. Их значения вводятся сообщением в виде диапазона («45-70»), нижней границы («от 2015») или одного числа. Символ «-» сбрасывает фильтр.

Затем бот просит указать цену в белорусских рублях. Это минимальная цена, по которой товар сейчас продаётся на рынке: бот будет присылать объявления дешевле неё. На следующем шаге бот показывает стоп-слова, по которым объявления отсеиваются автоматически (например, «чехол» для электроники), и предлагает добавить свои слова-исключения через запятую, например «на запчасти, разбит». Шаг можно пропустить.

В конце бот показывает сводку: название, категорию, фильтры, цену и исключения. После подтверждения товар сохраняется, и бот сразу ищет на площадке объявления дешевле указанной цены. До пяти самых дешёвых из них приходят отдельным сообщением. Если таких объявлений нет, бот сообщает об этом и продолжает следить за площадкой. Результат мгновенного поиска показан на рисунке 4.7.

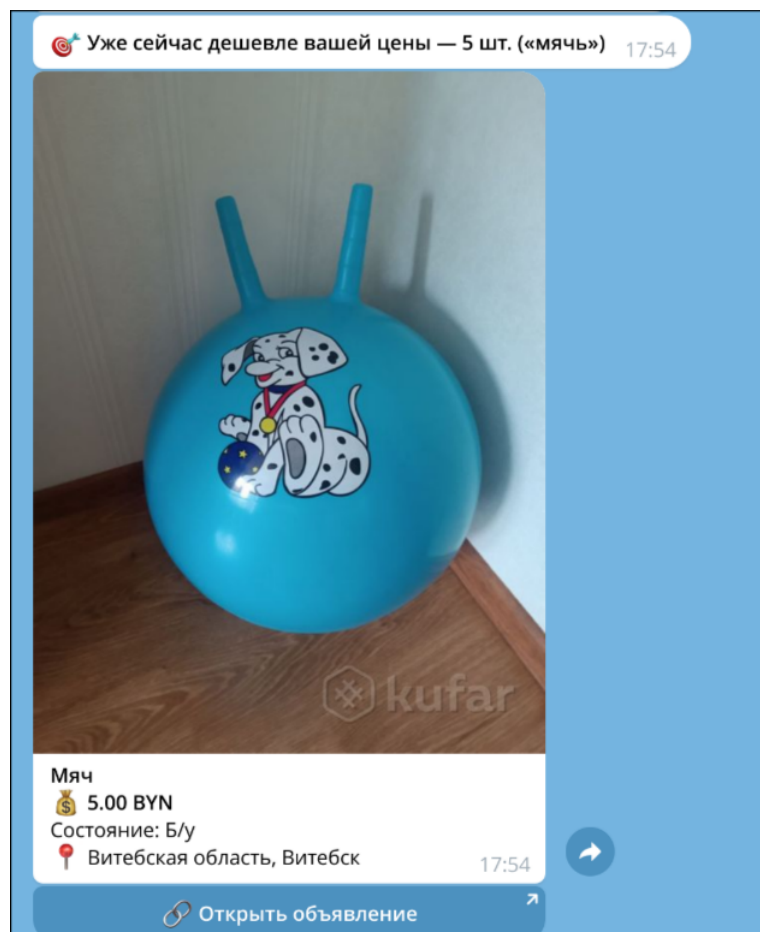


Рисунок 4.7 – Сохранение товара и мгновенный поиск

Добавить товар не получится, если занят личный лимит (3 товара) или общий лимит системы (25 товаров). Бот объясняет, какой из них достигнут. В первом случае нужно удалить ненужный товар в разделе «Мои товары», во втором остаётся подождать, пока место освободится.

На любом шаге диалог можно прервать кнопкой «Отмена», введённые данные при этом не сохраняются.

4.2.2 Импорт фильтров из ссылки

Если поиск уже настроен на сайте *Kufar.by*, его не нужно повторять в боте. Пользователь настраивает фильтры на сайте или открывает свой сохранённый поиск, копирует ссылку из адресной строки и отправляет её боту после нажатия «Вставить ссылку с *Kufar*». Бот переносит категорию, фильтры и цену и перечисляет неподдерживаемые параметры, например фильтр по станции метро. Цену из ссылки можно оставить одной кнопкой или ввести другую. Ссылку на страницу сохранённого поиска в личном кабинете бот принять не может, потому что её содержимое видно только владельцу, и подсказывает, какую ссылку прислать вместо неё.

4.2.3 Получение уведомлений

Когда на площадке появляется подходящее объявление, бот присылает сообщение. В заголовке указан тип события: «Новое предложение» или «Цена упала». В сообщении приведены цена (при снижении – старая зачёркнутая и новая), на сколько процентов цена ниже порога пользователя, состояние товара, местоположение, время публикации и ссылка «Открыть объявление». Первая фотография объявления идёт вместе с текстом, ещё до четырёх приходят следом альбомом. Пример уведомления показан на рисунке 4.8.

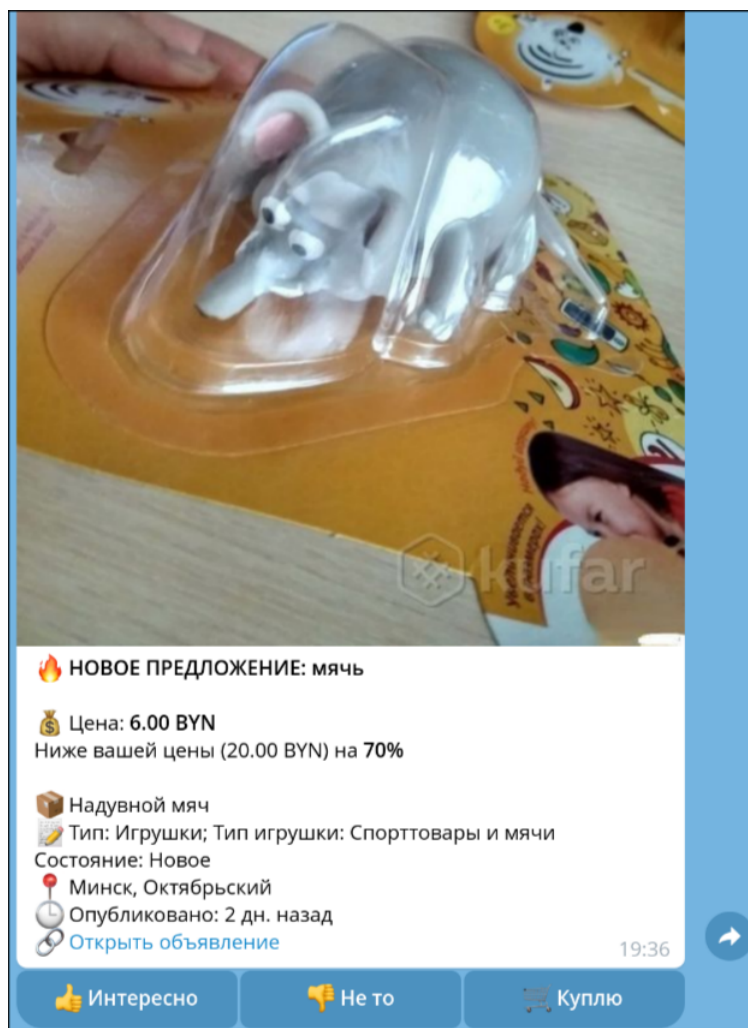


Рисунок 4.8 – Уведомление о выгодном предложении

Под уведомлением расположены кнопки «Интересно», «Не то» и «Куплю». Оценка сохраняется и попадает в отчёт, а число покупок по рекомендации бота видно в профиле. Одно и то же объявление повторно приходит только в том случае, если его цена стала ниже, чем в прошлом уведомлении. Чтобы уведомлений не было слишком много, по одному товару приходит не более 5

сообщений в час и не более 30 в сутки на пользователя; при большом числе находок первыми приходят самые дешёвые.

4.2.4 Управление списком товаров

Кнопка «Мои товары» или команда */my* открывает список отслеживаемых товаров. Над списком указано, сколько товаров отслеживается из личного лимита и сколько занято во всей системе. Под списком находятся кнопки «Экспорт CSV» и «Импорт CSV». Нажатие на товар открывает его карточку с категорией, ценой, фильтрами, статусом, словами-исключениями и автоматическими стоп-словами. Карточка товара показана на рисунке 4.9.

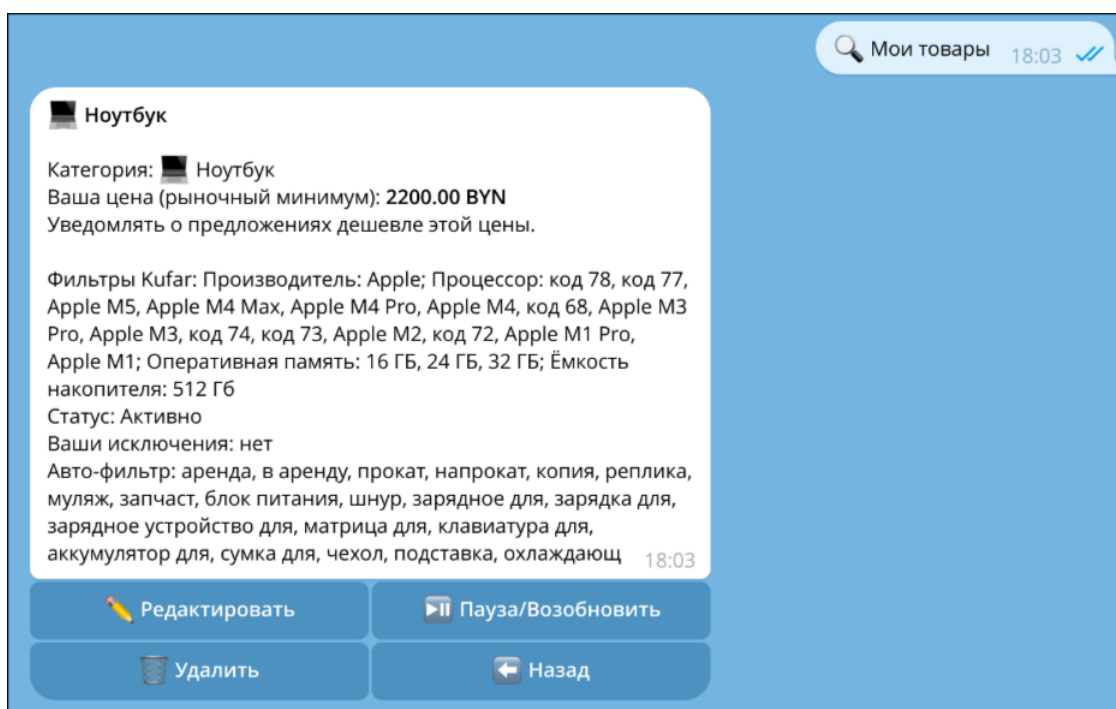


Рисунок 4.9 – Карточка отслеживаемого товара

В карточке доступны действия:

- «Редактировать» – изменить название, цену, слова-исключения, а в категориях с фильтрами площадки и сами фильтры;
- «Пауза/Возобновить» – временно остановить отслеживание без удаления товара;
- «Удалить» – прекратить отслеживание и удалить товар;
- «Назад» – вернуться к списку.

Кнопка «Экспорт CSV» присылает файл со списком товаров, который сразу открывается в *Excel* по столбцам. Файл можно отредактировать и загрузить обратно кнопкой «Импорт CSV», например чтобы перенести список на

другую учётную запись. При импорте бот проверяет каждую строку так же, как при ручном вводе, и сообщает, сколько товаров добавлено, какие строки пропущены и почему.

4.2.5 Настройки и электронная почта

Кнопка «Настройки» или команда */settings* показывает, включены ли уведомления, выбранный язык и состояние электронной почты. Здесь можно отключить и снова включить уведомления, сменить язык интерфейса и перейти в раздел «Почта».

В разделе «Почта» пользователь нажимает «Привязать адрес» и вводит адрес электронной почты. На него приходит шестизначный код, который действует 15 минут; его нужно ввести в бота. Если письмо не пришло, код можно запросить снова, но не чаще раза в минуту. После подтверждения копии уведомлений раз в 5 минут приходят на почту сводным письмом, а отчёты можно получать вложением. Копии отключаются кнопкой в том же разделе, там же адрес можно сменить или отвязать.

4.2.6 Профиль и отчёты

Команда */profile* показывает идентификатор, имя, статус, число отслеживаемых и активных товаров, найденных предложений, покупок по рекомендации и занятость лимита. Кнопка «Отчёт в *Excel*» под профилем или команда */report* предлагает выбрать период: 7, 30 или 90 дней. При подтверждённой почте появляются такие же кнопки для отправки отчёта письмом.

Отчёт состоит из двух листов. На листе «Находки» по одной строке на каждое найденное предложение: дата, товар, объявление, цена, порог, выгода в процентах, тип события, статус доставки, оценка и ссылка. Цены записаны числами, поэтому таблицу можно сортировать и фильтровать средствами *Excel*. На листе «Итоги» указаны период, число находок и доставленных уведомлений, средняя выгода и лучшая находка. Пример отчёта приведён на рисунке 4.10.

	А	В
1	Найденные предложения	
2		
3	Пользователь	misha_iosko
4	Период	последние 90 дн.
5	Сформирован	21.09.2026 15:05
6	Всего находок	3
7	Доставлено уведомлений	0
8	Средняя выгода, %	20.3
9	Лучшая находка	1-комнатная квартира Билево-1, на длительный срок — 750.00 BYN (-25.0%)

Рисунок 4.10 – Отчёт о найденных предложениях

4.2.7 Действия при ошибках

Бот отвечает на каждое действие. Если сервис временно недоступен, пользователь получает сообщение с просьбой повторить попытку через минуту. Если бот не понял сообщение, например диалог добавления прервался после перезапуска, пользователю предлагается начать заново кнопкой «Добавить». Команда */help* в любой момент показывает список команд и краткое описание работы бота.

ЗАКЛЮЧЕНИЕ

Цель курсового проекта достигнута: спроектирована и реализована автоматизированная информационная система отслеживания цен на товары в виде *Telegram*-бота. Система каждые 3 минуты опрашивает площадку *Kufar.by*, отбирает объявления дешевле заданной пользователем цены и уведомляет о новых предложениях и о снижении цены без повторов.

Анализ предметной области показывает, что ручной мониторинг не успевает за круглосуточным появлением объявлений, а существующие средства решают лишь часть задачи: сохранённые поиски площадки не отслеживают цену во времени, зарубежные трекеры рассчитаны на новые товары с артикулом, открытые боты ограничиваются текстовым поиском.

Ядро работы – база данных под управлением *PostgreSQL*. *ER*-модель в нотации Питера Чена из семи сущностей переходит в логическую и физическую схемы. Схема отвечает третьей нормальной форме, а отступления сделаны осознанно: *JSON*-поля хранят фильтры категорий, журнал уведомлений – цену на момент отправки для проверки повторов. Индексы подобраны под запросы цикла опроса и доставки, прежде всего под поиск объявления по внешнему идентификатору – более 500 тысяч раз в сутки. Целостность поддерживают внешние ключи, ограничения уникальности и перечислимые типы.

Программная часть разделена на серверный сервис на *FastAPI*, который выполняет *SQL*-запросы и периодические задачи, и бота на *aiogram* для диалога с пользователем. Отбор по бренду, размеру рамы или числу комнат выполняет сама площадка. Система реализует весь функционал из постановки задачи: импорт фильтров из ссылки, мгновенный поиск, обмен списком товаров через *CSV*, отчёты в *Excel*, два языка интерфейса и копии уведомлений на почту.

Надёжность опирается на свойства СУБД. Найденное совпадение сначала фиксируется в базе данных, а отправляет его отдельная задача с повтором до пяти попыток; отметка о доставке и запись в журнал сохраняются одной транзакцией. Работу системы подтверждают более 280 автоматических тестов.

Система допускает развитие: модель данных предусматривает несколько площадок, поэтому к *Kufar.by* добавляются *Onliner*, *av.by* и другие площадки. Система решает реальную задачу покупателей и перекупщиков: внешне это бот с кнопками, по существу – система обработки потока данных, качество которой определяет проектирование базы данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] *Kufar.by* – площадка объявлений [Электронный ресурс]. – Режим доступа: <https://www.kufar.by>. – Дата доступа: 19.09.2026.
- [2] *CamelCamelCamel – Amazon price tracker* [Электронный ресурс]. – Режим доступа: <https://camelcamelcamel.com>. – Дата доступа: 19.09.2026.
- [3] *Keepa – Amazon price tracker* [Электронный ресурс]. – Режим доступа: <https://keepa.com>. – Дата доступа: 19.09.2026.
- [4] *Chen, P. P. The Entity-Relationship Model – Toward a Unified View of Data / P. P. Chen // ACM Transactions on Database Systems. – 1976. – Vol. 1, No. 1. – P. 9–36.*
- [5] *Brown, S. The C4 model for visualising software architecture* [Электронный ресурс]. – Режим доступа: <https://c4model.com>. – Дата доступа: 19.09.2026.
- [6] *Telegram Bot API* [Электронный ресурс]. – Режим доступа: <https://core.telegram.org/bots/api>. – Дата доступа: 19.09.2026.
- [7] Документация к *PostgreSQL 16* [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql/16>. – Дата доступа: 19.09.2026.
- [8] *asyncpg Documentation* [Электронный ресурс]. – Режим доступа: <https://magicstack.github.io/asyncpg/>. – Дата доступа: 19.09.2026.
- [9] *Alembic Documentation* [Электронный ресурс]. – Режим доступа: <https://pypi.org/project/alembic>. – Дата доступа: 19.09.2026.
- [10] *FastAPI Documentation* [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com>. – Дата доступа: 19.09.2026.
- [11] *aiogram 3 Documentation* [Электронный ресурс]. – Режим доступа: <https://docs.aiogram.dev>. – Дата доступа: 19.09.2026.
- [12] *APScheduler Documentation* [Электронный ресурс]. – Режим доступа: <https://apscheduler.readthedocs.io>. – Дата доступа: 19.09.2026.
- [13] *Docker Compose Documentation* [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/compose/>. – Дата доступа: 19.09.2026.
- [14] СТП 01-2024. Стандарт предприятия. Дипломные проекты (работы). Общие требования. – Минск: БГУИР, 2024. – 169 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг основных функций программного модуля

— *Database schema, PostgreSQL 16*

```
CREATE TYPE platform AS ENUM ('KUFAR', 'ONLINER', 'DEAL_BY',  
    'WILDBERRIES', 'OZ_BY', 'TWENTYONE_VEK', 'AV_BY');  
  
CREATE TYPE productcategory AS ENUM ('ELECTRONICS', 'TOOLS',  
    'FURNITURE', 'CLOTHING', 'AUTO', 'SPORTS', 'HOUSEHOLD',  
    'KIDS', 'GARDEN', 'OTHER', 'BICYCLE', 'LAPTOP', 'APARTMENT');  
  
CREATE TYPE conditionpreference AS ENUM ('ANY', 'WORKING',  
    'LIKE_NEW');  
  
CREATE TYPE flexibilitylevel AS ENUM ('STRICT', 'MODERATE',  
    'FLEXIBLE');  
  
CREATE TYPE attributetype AS ENUM ('TEXT', 'NUMBER', 'BOOLEAN',  
    'RANGE');  
  
CREATE TYPE attributepriority AS ENUM ('MUST_HAVE',  
    'NICE_TO_HAVE', 'INFORMATIONAL');  
  
CREATE TYPE userreaction AS ENUM ('INTERESTED',  
    'NOT_INTERESTED', 'BOUGHT');  
  
CREATE TABLE kufar_attributes (  
    id SERIAL NOT NULL,  
    category_id INTEGER NOT NULL,  
    p VARCHAR(100) NOT NULL,  
    pu VARCHAR(20) NOT NULL,  
    label VARCHAR(255) NOT NULL,  
    seen_count INTEGER NOT NULL,  
    updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now()  
        NOT NULL,  
    PRIMARY KEY (id),
```

```

    CONSTRAINT uq_kufar_attr UNIQUE (category_id, p)
);

CREATE INDEX ix_kufar_attributes_category_id ON kufar_attributes
    (category_id);

CREATE TABLE listings (
    id SERIAL NOT NULL,
    platform platform NOT NULL,
    external_id VARCHAR(255) NOT NULL,
    title TEXT NOT NULL,
    description TEXT,
    price_byn FLOAT,
    currency VARCHAR(10) NOT NULL,
    url TEXT NOT NULL,
    images JSON NOT NULL,
    location VARCHAR(255),
    seller_info JSON,
    condition VARCHAR(50),
    raw_data JSON,
    posted_at TIMESTAMP WITHOUT TIME ZONE,
    first_seen_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now()
        NOT NULL,
    last_seen_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now()
        NOT NULL,
    PRIMARY KEY (id)
);

CREATE INDEX ix_listings_external_id ON listings (external_id);
CREATE INDEX ix_listings_platform_external_id ON listings (platform,
    external_id);

CREATE TABLE price_references (
    id SERIAL NOT NULL,
    product_query VARCHAR(255) NOT NULL,
    platform platform NOT NULL,
    min_price FLOAT,
    avg_price FLOAT,

```

```
max_price FLOAT,
sample_count INTEGER NOT NULL,
fetched_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT NULL,
PRIMARY KEY (id)
);
```

```
CREATE INDEX ix_price_references_product_query ON price_references
(product_query);
```

```
CREATE TABLE users (
id SERIAL NOT NULL,
telegram_id BIGINT NOT NULL,
username VARCHAR(255),
first_name VARCHAR(255),
language VARCHAR(10) NOT NULL,
is_active BOOLEAN NOT NULL,
notification_enabled BOOLEAN NOT NULL,
email VARCHAR(255),
email_verified BOOLEAN DEFAULT false NOT NULL,
email_notifications BOOLEAN DEFAULT false NOT NULL,
email_code_hash VARCHAR(64),
email_code_expires_at TIMESTAMP WITHOUT TIME ZONE,
email_code_sent_at TIMESTAMP WITHOUT TIME ZONE,
email_code_attempts INTEGER DEFAULT 0 NOT NULL,
created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT NULL,
updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT NULL,
PRIMARY KEY (id)
);
```

```
CREATE UNIQUE INDEX ix_users_telegram_id ON users (telegram_id);
```

```
CREATE TABLE kufar_attribute_values (
id SERIAL NOT NULL,
attribute_id INTEGER NOT NULL,
value_code VARCHAR(50) NOT NULL,
```

```

value_label VARCHAR(255) NOT NULL,
parent_code VARCHAR(50),
seen_count INTEGER NOT NULL,
updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now()
NOT NULL,
PRIMARY KEY (id),
CONSTRAINT uq_kufar_attr_value UNIQUE (attribute_id, value_code),
FOREIGN KEY(attribute_id) REFERENCES kufar_attributes (id) ON
DELETE CASCADE
);

```

```

CREATE INDEX ix_kufar_attribute_values_attribute_id ON
kufar_attribute_values (attribute_id);

```

```

CREATE TABLE tracked_products (
id SERIAL NOT NULL,
user_id INTEGER NOT NULL,
name VARCHAR(255) NOT NULL,
category productcategory NOT NULL,
min_price_byn FLOAT NOT NULL,
max_price_byn FLOAT,
condition_preference conditionpreference NOT NULL,
flexibility flexibilitylevel NOT NULL,
search_keywords JSON NOT NULL,
exclude_keywords JSON NOT NULL,
kufar_filters JSON NOT NULL,
min_deal_score FLOAT NOT NULL,
is_active BOOLEAN NOT NULL,
created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT
NULL,
PRIMARY KEY (id),
FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE
CASCADE
);

```

```

CREATE INDEX ix_tracked_products_user_id ON tracked_products (user_id);

```

```

CREATE TABLE custom_attributes (

```

```

id SERIAL NOT NULL,
tracked_product_id INTEGER NOT NULL,
attribute_name VARCHAR(100) NOT NULL,
attribute_type attributetype NOT NULL,
value TEXT NOT NULL,
priority attributepriority NOT NULL,
PRIMARY KEY (id),
FOREIGN KEY(tracked_product_id) REFERENCES tracked_products (id)
ON DELETE CASCADE
);

```

```

CREATE INDEX ix_custom_attributes_tracked_product_id ON
  custom_attributes (tracked_product_id);

```

```

CREATE TABLE matches (
  id SERIAL NOT NULL,
  tracked_product_id INTEGER NOT NULL,
  listing_id INTEGER NOT NULL,
  deal_score FLOAT NOT NULL,
  match_details JSON NOT NULL,
  notified_at TIMESTAMP WITHOUT TIME ZONE,
  notify_attempts INTEGER DEFAULT 0 NOT NULL,
  notify_error VARCHAR(255),
  email_notified_at TIMESTAMP WITHOUT TIME ZONE,
  email_attempts INTEGER DEFAULT 0 NOT NULL,
  user_reaction userreaction,
  created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT
    NULL,
PRIMARY KEY (id),
FOREIGN KEY(tracked_product_id) REFERENCES tracked_products (id)
ON DELETE CASCADE,
FOREIGN KEY(listing_id) REFERENCES listings (id) ON DELETE
CASCADE
);

```

```

CREATE INDEX ix_matches_listing_id ON matches (listing_id);
CREATE INDEX ix_matches_tracked_product_id ON matches
  (tracked_product_id);

```

```

CREATE TABLE sent_notifications (
  id SERIAL NOT NULL,
  user_id INTEGER NOT NULL,
  tracked_product_id INTEGER NOT NULL,
  listing_id INTEGER NOT NULL,
  match_id INTEGER,
  listing_external_id VARCHAR(255) NOT NULL,
  platform platform NOT NULL,
  price_byn FLOAT,
  market_avg_byn FLOAT,
  discount_pct FLOAT,
  deal_score FLOAT,
  sent_at TIMESTAMP WITHOUT TIME ZONE DEFAULT now() NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE,
  FOREIGN KEY(tracked_product_id) REFERENCES tracked_products (id) ON DELETE CASCADE,
  FOREIGN KEY(listing_id) REFERENCES listings (id),
  FOREIGN KEY(match_id) REFERENCES matches (id) ON DELETE SET NULL
);

```

```

CREATE INDEX ix_sent_notifications_listing_external_id ON
  sent_notifications (listing_external_id);
CREATE INDEX ix_sent_notifications_tracked_product_id ON
  sent_notifications (tracked_product_id);
CREATE INDEX ix_sent_notifications_user_id ON sent_notifications
  (user_id);

```

*Файл vedomost.pdf не найден. Пожалуйста, добавьте его для включения
ведомости в документ.*